



Javascript Basics

Javascript Basics	6
Chapter 01: JS Basics	6
■ What is Javascript?	6
✓ Is JavaScript Compiled or Interpreted?	9
■ NODE js in simple terms	11
■ Install Visual Studio Code	12
Install Node.js	12
■ Install VS Code - Visual Studio Code (	13
■ How JavaScript Engine Works	15
Overview Flow	15
■ Step-by-Step Breakdown	16
1. Lexical Analysis (Tokenizing)	16
2. Parsing (Syntax Analysis)	17
3. Interpreter (Generates Bytecode)	17
4. Execution	18
5. JIT Compilation (Optimization)	18
6. De-optimization	19
Summary	19
■ How does a V8 engine Works?	20
Step-by-Step Breakdown	22
Step 1: Scanner (Lexical Analysis / Tokenizing)	22
Step 2: Parser (Syntax Analysis)	23
Step 3: Ignition (Interpreter)	24
Step 4 : Execution + Profiling	24
Step 5 : TurboFan (JIT)	25
Step 6: De-optimization (Bailout)	26
Step 7: Garbage Collection:	27
ECMAScript Standard	29
What is ECMAScript?	29
Chapter 02: Variables (var, let, const)	31
■ Variables in JavaScript: Comments, Identifiers, Keywords, Literals, var, let, const	31
■ What are Variables?	31
■ Comments in JavaScript	31
Single Line Comment	31
Multi Line Comment	31
When to use Comments	32
■ Identifier, Literal, and Operator	32



Chapter 03: Identifiers, Literals & Operators	33
■ What is a Keyword?	33
■ What is a Literal?	33
■ What is an Identifier?	33
■ Rules of Identifier	34
Valid Identifier Examples	34
Case Sensitivity Example	34
Unicode Identifiers (Yes, JavaScript supports them!)	35
■ var (Function Scoped)	35
Example 1: Global Scope	35
Example 2: var inside a Function	35
Why does console.log("F ->", a) print 30?	36
Example 3: var allows Re-declaration	36
Key Points about var	36
■ let (Block Scoped)	36
Example 1: Global Scope	36
Example 2: let inside a Function	36
Why does console.log("let ->", b) print 30?	37
Example 3: let does NOT allow Re-declaration in the same scope	37
Example 4: let allows Reassignment	37
Example 5: Cannot mix var and let for the same name	38
Key Points about let	38
■ const (Block Scoped + Constant)	38
Example 1: Declaring a constant	39
Key Points about const	39
var vs let vs const: Quick Comparison	39
■ var vs let: Scope Difference (Visual)	40
■ When to Use What?	40
Chapter 04: Operators	41
■ Operators in JavaScript	41
■ Arithmetic Operators	41
■ Assignment Operators	42
■ Comparison Operators	42
■ Logical Operators	43
■ String Operators	44
■ Ternary Operator	44
■ typeof Operator	45
■ Nullish Coalescing Operator (??)	45
Chapter 05: Statements (if / else)	46
■ What is a Statement?	46
■ The if / else Statement	46
■ The else if Ladder	47
■ Nested if (Real Live Example: User Roles)	47
■ if / else with API Status Codes	48



■ Truthy and Falsy Values (Interview Favourite)	49
■ Logical Operators inside if	50
■ if without Curly Braces	51
■ Interview Questions (Common Traps)	51
Chapter 06: Switch Statements	52
■ The switch Statement	52
■ Why break Matters (Fall-Through)	53
■ The default Case	54
■ Real Example: API Response Codes	55
■ Grouping Cases (Shared Logic)	56
■ The switch(true) Pattern (Ranges)	57
■ Interview Questions (Common Traps)	58
Chapter 07: Loops	60
■ Why Do We Need Loops?	60
■ Increment and Decrement Operators	60
■ The for Loop	61
■ The while Loop	62
■ The do...while Loop	63
Chapter 08: Arrays	64
■ What is an Array?	64
■ Creating Arrays	65
■ Accessing and Modifying	65
■ Adding and Removing Items	66
■ Real Example: Browser List	67
■ Searching in Arrays	68
■ Iterating (Looping) Over Arrays	68
■ Transforming Arrays (map, filter, reduce)	70
■ Sorting Arrays	70
■ Slicing and Combining	71
■ Checking Arrays	72
■ Copying Arrays (Shallow vs Reference)	72
■ Array Destructuring	73
Chapter 09: Functions	74
■ Why Do We Need Functions?	74
■ The 4 Types of Functions	74
■ Function Expression	76
■ Arrow Functions (ES6)	76
■ IIFE (Immediately Invoked Function Expression)	77
■ Default, Rest, and Spread Parameters	78
■ Return Values	79
■ Hoisting	80
■ Scope in Functions	80
■ Closures	81
■ Higher-Order Functions	83



■ Callback Functions	83
■ Pure Functions	84
Chapter 10: Strings	85
■ What is a String?	85
■ String Properties and Access	86
■ Searching and Checking	86
■ Extracting Substrings	87
■ Transforming Strings	88
■ String Conversion and Immutability	89
Chapter 11: Objects	90
■ What is an Object?	90
■ Accessing and Modifying Properties	91
■ Primitives vs References (Very Important)	92
■ Object Methods and this	93
■ Object Destructuring	94
■ Spread and Merging Objects	94
■ Getters and Setters	95
■ Looping Over Objects	95
■ Real Example: Config Objects	96
■ let vs const with Objects	97
Chapter 12: Multi-Dimensional Arrays	98
■ What is a Multi-Dimensional Array?	98
■ Accessing and Modifying (row, col)	98
■ Looping Over a 2D Array	99
■ Common 2D Operations	100
■ Star Patterns (Interview Practice)	101
Chapter 13: Callbacks	103
■ What is a Callback?	103
■ Three Ways to Pass a Callback	104
■ Callbacks with Parameters	105
■ Synchronous Callbacks	105
■ Asynchronous Callbacks	105
■ Callback Hell (Pyramid of Doom)	106
Chapter 14: Promises	108
■ What is a Promise?	108
■ .then() — Handling Success	109
■ .catch() — Handling Errors	109
■ .finally() — Always Runs	109
■ Promise Chaining (Real Login Flow)	110
■ Promise.all — Wait for All (Fail Fast)	111
■ Promise.allSettled — Wait for All (Never Fails)	112
■ Promise.race — First One Wins	113
Chapter 15: Async / Await	114
■ What is async / await?	114



■ From .then() to async / await	115
■ Error Handling with try / catch	116
■ Sequential Execution	116
■ Parallel Execution with Promise.all	117
■ Real QA Example: Retry a Flaky Test	118
Chapter 16: OOP (Classes and Modules)	120
■ Export and Import (Modules)	120
■ What is a Class?	120
■ Constructor and Objects	121
■ Public and Private Fields (#)	123
■ Static Members	123
■ Encapsulation (Getters and Setters)	124
Chapter 17: OOP Inheritance	126
■ What is Inheritance?	126
■ The super Keyword	127
■ Method Overriding	128
■ Polymorphism	129
■ Multi-Level Inheritance	131
■ Hierarchical Inheritance	132
■ Multiple Inheritance (via Mixins)	132
■ Exporting Classes (Page Object Model)	133
Chapter 18: TypeScript Basics	135
■ What is TypeScript?	135
■ Basic Types	135
■ any vs unknown	136
■ Function and Object Annotations	136
■ Special Return Types: void and never	137
■ Real QA Examples	137
Chapter 19: TypeScript Interfaces	139
■ What is an Interface?	139
■ Optional (?) and readonly Properties	140
■ Method Signatures in an Interface	141
■ Function-Type Interfaces (Test Hooks)	141
■ Extending Interfaces	142
■ A Class that implements an Interface	143
■ Index Signatures	144
■ Real Example: Test Config and Bug Report	144
Chapter 20: TypeScript Enums	146
■ What is an Enum?	146
■ Numeric Enums and Reverse Mapping	146
■ String Enums for Configuration	147
■ Real Example: Browser Selection	147
■ Real Example: HTTP Methods	149
Chapter 21: TypeScript Generics	150



■ What are Generics?	150
■ Generic Classes	150
■ Real Example: Generic API Response	152
Chapter 22: TypeScript Access Modifiers	153
■ Access Modifiers: public, private, protected	153
■ readonly Properties	155
■ Abstract Classes	156
■ Type Assertion (as)	156
■ The override Keyword	157
■ Decorators	158
Chapter 23: Playwright Fundamentals	160
■ What is Playwright?	160
■ The Config File	160
■ Your First Test	161
■ Actions and Locators	162
■ How It All Connects	162
■ Running Tests	163

Chapter 01: JS Basics

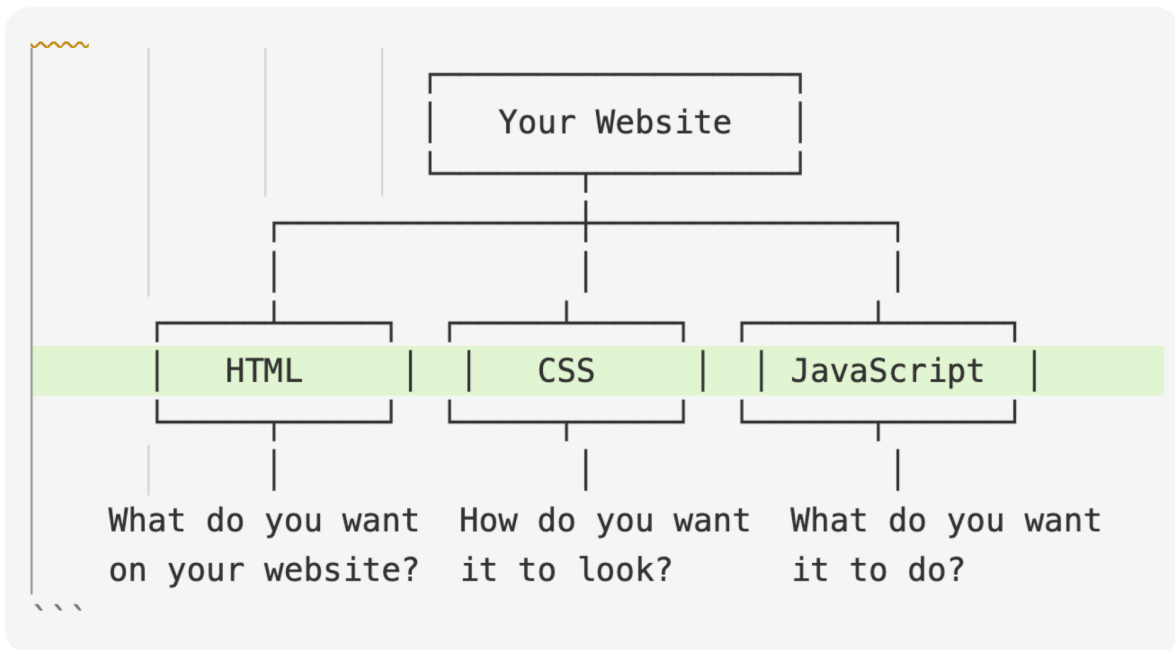
■ What is Javascript?

- JavaScript is a **high-level, interpreted programming language primarily** used to make web pages interactive and dynamic. It was created by Brendan Eich in 1995 at Netscape.
- Where it runs: ***Originally designed to run in web browsers,*** JavaScript now also runs on servers (via Node.js), mobile apps, desktop applications, and even IoT devices.
- Core features: **It supports object-oriented, functional, and event-driven programming styles.** It's dynamically typed, meaning you don't need to declare variable types explicitly.



- ECMAScript — mention that JavaScript follows the ECMAScript standard (ES6/ES2015+)
- Role in web development: It's **one of the three core** technologies of the web alongside HTML (structure) and CSS (styling). JavaScript handles the behavior and logic layer.

HTML Structure / Content
CSS Styling / Appearance
JavaScript Behavior / Interactivity



- The DOM: JavaScript can manipulate the Document Object Model (DOM), allowing it to dynamically change a webpage's content, structure, and styling in real time without reloading.
- Asynchronous programming: It supports asynchronous operations through callbacks, Promises, and `async/await`, making it efficient for tasks like API calls and file handling.
- Ecosystem: It has a massive ecosystem of libraries and frameworks such as React, Angular, Vue.js (frontend) and Express, Next.js (backend), supported by the npm package manager.
- Universal adoption: JavaScript is the most widely used programming language in the world, supported by all modern browsers and backed by a huge developer community.



✓ Is JavaScript Compiled or Interpreted?

JavaScript is an interpreted language with JIT compilation at runtime.

Browser	JS Engine	Developer
Google Chrome	V8	Google
Microsoft Edge	V8	Google (Chromium)
Mozilla Firefox	SpiderMonkey	Mozilla
Apple Safari	JavaScriptCore (Nitro)	Apple
Opera	V8	Google (Chromium)
Brave	V8	Google (Chromium)
Samsung Internet	V8	Google (Chromium)
Node.js (runtime)	V8	Google
Deno (runtime)	V8	Google
Bun (runtime)	JavaScriptCore	Oven Technologies
Internet Explorer (legacy)	Chakra	Microsoft

Screenshot by Znapper.com

✓ JavaScript: Compile-Time & Runtime?

- JavaScript is Primarily a Runtime (Interpreted) Language
Traditionally, JavaScript code is executed line by line at runtime by the browser's JS engine.
- There's no separate compilation step like in C++ or Java.



- Simple Terms - JavaScript is a **runtime language** that uses **JIT compilation internally** for performance.
- So it's **not compiled ahead of time** like C++, but it's **not purely interpreted** either it sits somewhere **in between**.

✓ What is [Node.js](#)?

- Node.js is an open-source, cross-platform JavaScript runtime environment that allows you to run JavaScript code outside of a web browser primarily on servers.
- It was created by Ryan Dahl in 2009 and is built on Google Chrome's V8 JavaScript engine.

Key Features

Feature	Description
Server-Side JS	Allows JavaScript to run on servers, not just browsers
V8 Engine	Uses Google's fast V8 engine for high performance
Non-Blocking I/O	Handles multiple requests asynchronously without waiting
Single-Threaded	Uses a single thread with an event loop for concurrency
Event-Driven	Responds to events (requests, file reads) efficiently
NPM (Node Package Manager)	World's largest ecosystem of open-source libraries
Cross-Platform	Runs on Windows, macOS, and Linux

Screenshot by Snapper.com

Common Use Cases

- Web Servers & APIs (REST, GraphQL)
- Real-Time Apps (Chat apps, live notifications)
- Microservices Architecture
- Command-Line Tools (CLI apps)



- Streaming Applications (Video/audio)
- IoT (Internet of Things)

Popular Frameworks Built on Node.js

Framework	Purpose
Express.js	Minimal web server framework
Next.js	Full-stack React framework
NestJS	Enterprise-grade backend apps
Fastify	High-performance web framework
Socket.io	Real-time communication

Screenshot by Xnapper.com

NODE js in simple terms

Node.js = JavaScript + V8 Engine + Server-Side Capabilities,
enabling full-stack development using a single language.



Install Visual Studio Code

Visual Studio Code, and adding Node.js to the environment variables for Windows, macOS, and Linux.

Install Node.js

Windows:

1. Download the Windows installer from the [Node.js website](https://nodejs.org/).
2. Run the installer.
3. Follow the prompts in the installer (Accept the license agreement, click the NEXT button a bunch of times and finally install).
4. Restart your computer to ensure changes take effect.

MacOS:

1. Download the macOS installer from the [Node.js website](https://nodejs.org/).
2. Open the `.pkg`` file you downloaded and follow the instructions.
3. You may need to enter your admin password.
4. To verify installation, open a terminal and type ``node -v``.

Linux

1. Use a package manager to install Node.js.
 - For Ubuntu/Debian, use ``sudo apt-get install nodejs npm``.
 - For Fedora, use ``sudo dnf install nodejs``.
 - For other distributions, check the package manager or Node.js website.
2. Verify the installation with ``node -v`` and ``npm -v``.



Install VS Code - Visual Studio Code (

Windows, macOS, and Linux:

1. Go to the VS Code website, <https://code.visualstudio.com> and download the installer for your OS.
2. Run the installer.
3. Follow the installation instructions.
4. Launch VS Code after installation.

Adding Node.js to the Environment Variables

Windows:

1. Search for "Environment Variables" in the Start menu and open "Edit the system environment variables."
2. Click on "Environment Variables."
3. Under "System Variables," find the "Path" variable and click "Edit."
4. Click "New" and add the path to your Node.js installation (usually C:\Program Files\nodejs).
5. Click OK to save your changes.

macOS and Linux:

1. Open your terminal.
2. Edit your shell profile file (.bash_profile, .bashrc, .zshrc, etc.). For bash, use nano ~/.bash_profile or nano ~/.bashrc. For zsh, use nano ~/.zshrc.
3. Add export PATH="/usr/local/share/npm/bin:\$PATH" to your profile (adjust the path if your Node.js installation is different).
4. Save the file and reload your profile with source ~/.bash_profile, source ~/.bashrc, or source ~/.zshrc.



Install Code Runner

<https://open-vsx.org/extension/formulahendry/code-runner>

ESLint : Catches errors and enforces clean code

<https://marketplace.visualstudio.com/items?itemName=dbaeumer.vscode-eslint>

Prettier : Auto-formats your code perfectly

<https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>

Live Server : Live reload for instant browser preview

<https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer>

Error Lens : Shows errors/warnings directly in your code

<https://marketplace.visualstudio.com/items?itemName=usernamehw.errorlens>

JavaScript (ES6) : Code Snippets Speeds up coding with ready-made snippets

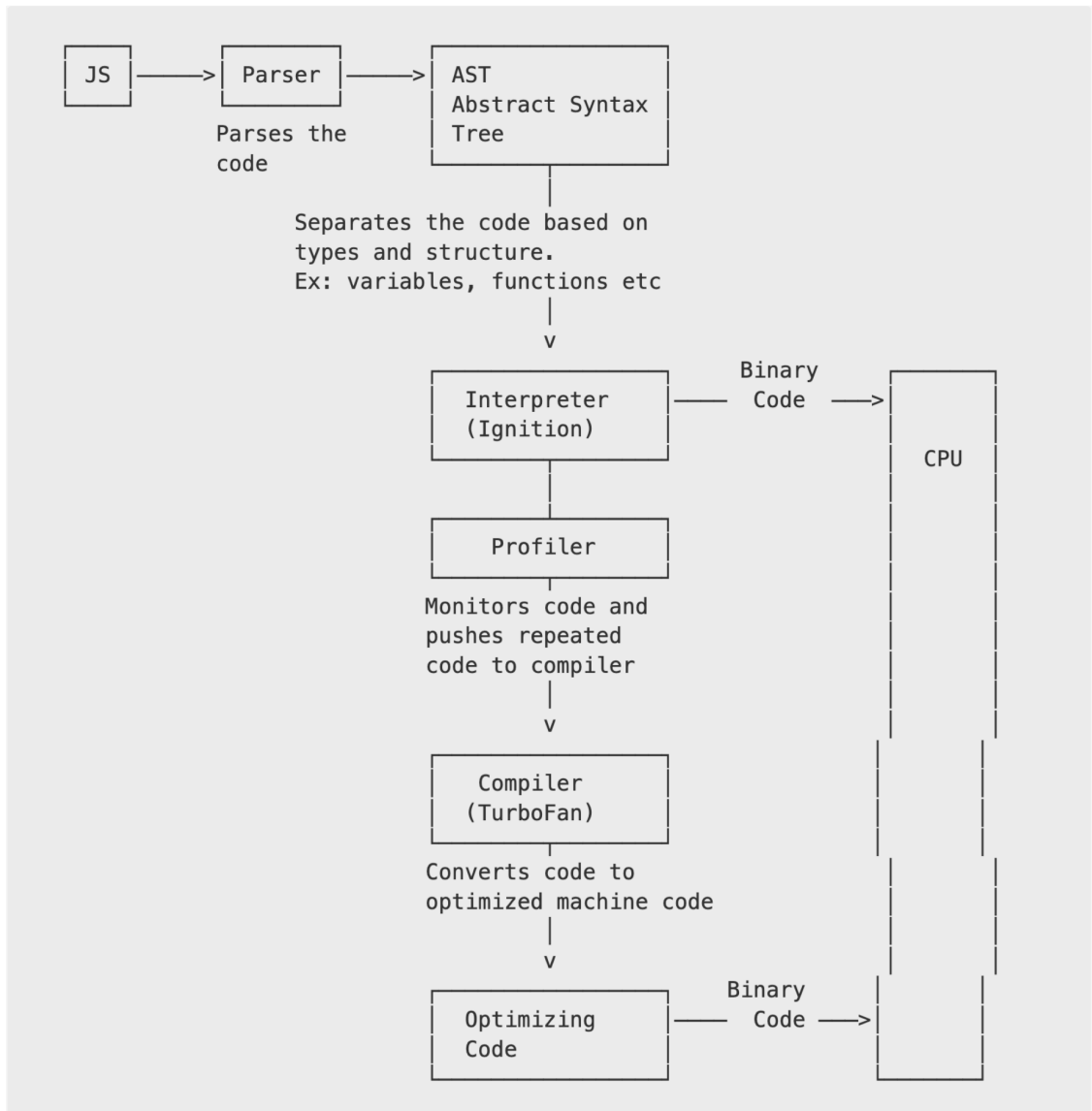
<https://marketplace.visualstudio.com/items?itemName=xabikos.JavaScriptSnippets>



How JavaScript Engine Works

A JavaScript engine is a program that **reads, interprets, and executes** JavaScript code. Here's a complete breakdown of how it works step by step.

Overview Flow



Screenshot by Xnapper.com



Step-by-Step Breakdown

1. Lexical Analysis (Tokenizing)

The engine first breaks the raw code into small meaningful chunks called **tokens**.

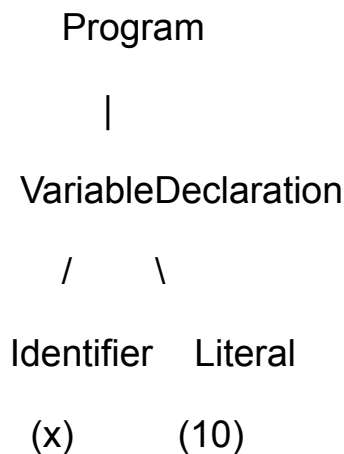
Example: `let x = 10;`

Tokens → `[let] [x] [=] [10] [;]`



2. Parsing (Syntax Analysis)

The tokens are then organized into a tree-like structure called an **Abstract Syntax Tree (AST)**. This represents the grammatical structure of the code.



If there's a **syntax error**, it gets caught at this stage.

3. Interpreter (Generates Bytecode)

The AST is passed to the **interpreter**, which converts it into **bytecode** — a lower-level, platform-independent code that the engine can execute quickly. This allows **fast startup** since the code doesn't need to be fully compiled first.



Engine	Interpreter Name
V8 (Chrome)	Ignition
SpiderMonkey (Firefox)	Baseline
JavaScriptCore (Safari)	LLInt

4. Execution

The bytecode is executed immediately. At this point, your JavaScript code is **running**.

Engine	JIT Compiler Name
V8 (Chrome)	TurboFan
SpiderMonkey (Firefox)	IonMonkey
JavaScriptCore (Safari)	FTL (Faster Than Light)

5. JIT Compilation (Optimization)

While the code runs, the engine **monitors** which parts of the code are used frequently (called "**hot code**").

The **JIT (Just-In-Time) Compiler** takes that hot code and compiles it into highly **optimized machine code** for faster execution.



6. De-optimization

If the engine's assumptions about the code turn out to be wrong (for example, a variable's type changes unexpectedly), it **de-optimizes** — throws away the optimized machine code and falls back to the bytecode version.

Example:

```
function add(a, b) { return a + b; }
```

```
add(5, 10);           // Engine assumes numbers → optimizes
```

```
add("hello", 5);      // Type changed! → De-optimizes
```

Key Components Inside the Engine

Component	Role
Call Stack	Tracks function execution (one at a time, single-threaded)
Memory Heap	Stores objects, variables, and data
Garbage Collector	Automatically frees unused memory
Event Loop	Handles async operations (not part of engine itself, provided by runtime)

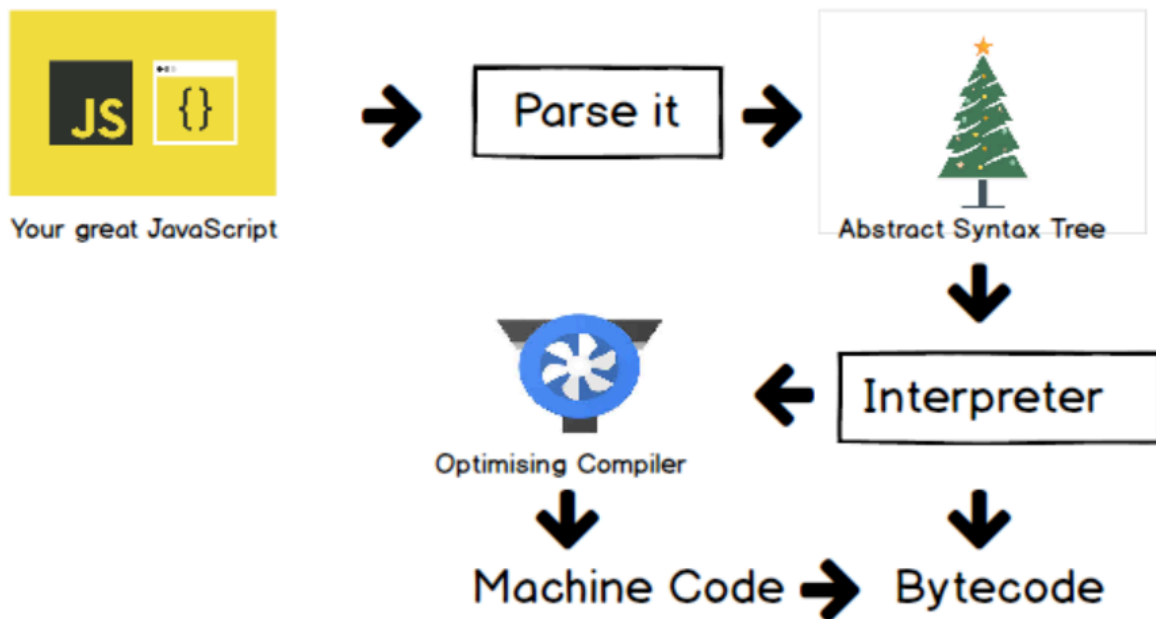
Summary

The JavaScript engine **parses** your code into a tree, **interprets** it into bytecode for quick execution, and then **JIT compiles** frequently used code into optimized

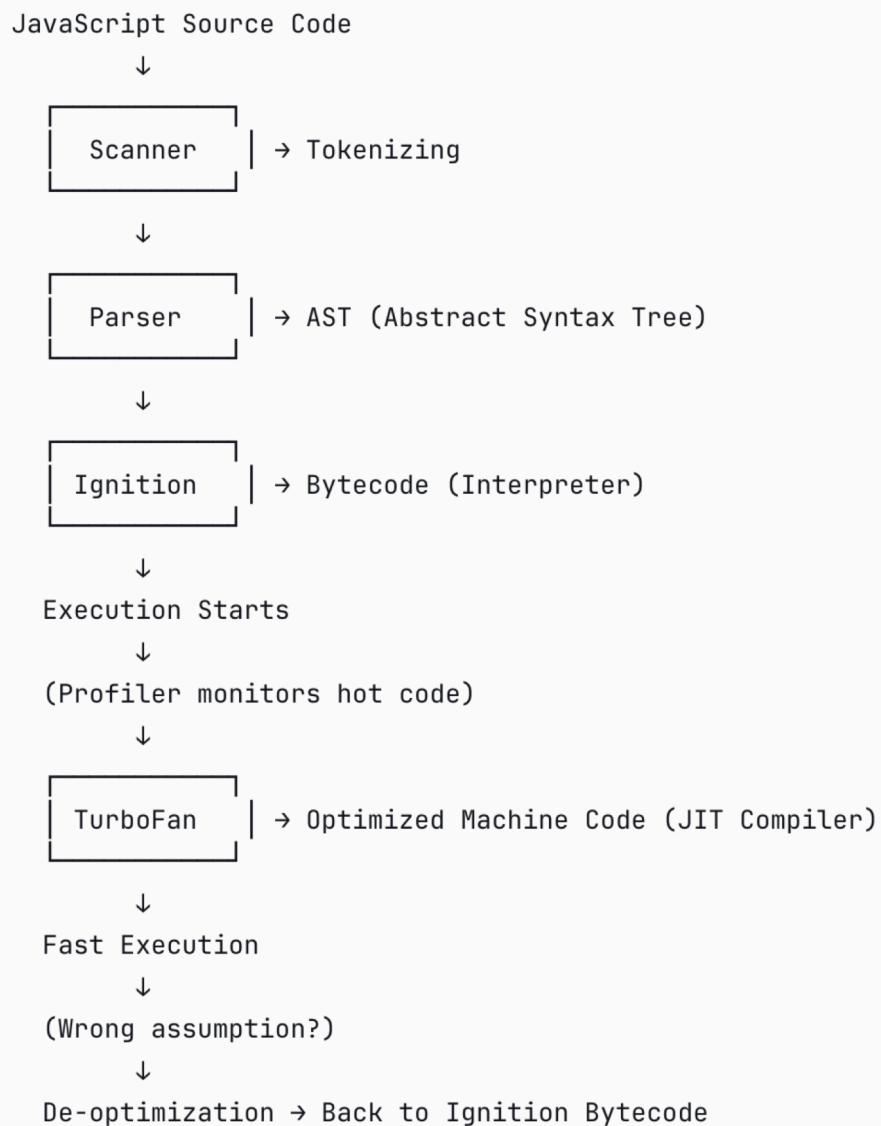
machine code for maximum speed. If assumptions fail, it **de-optimizes** and falls back.

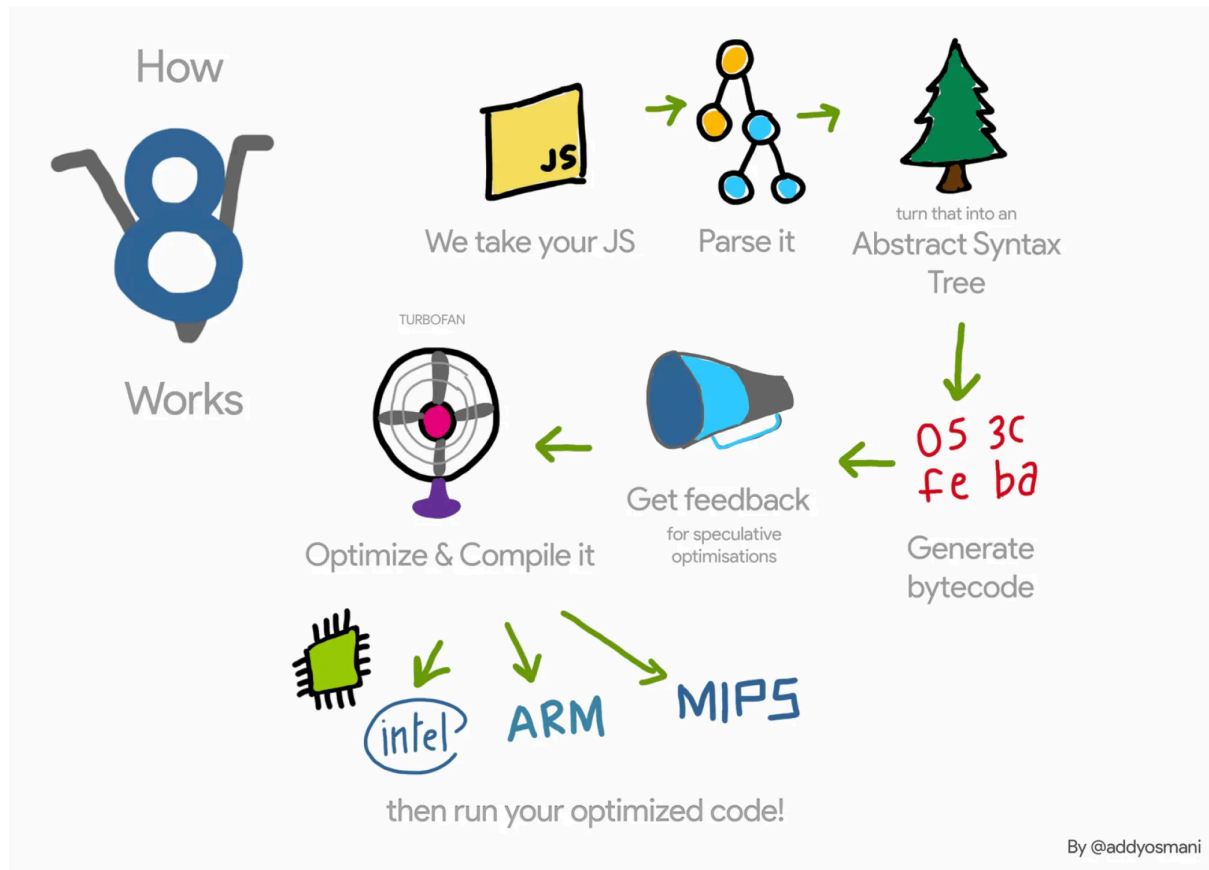
How does a V8 engine Works?

V8 is Google's open-source high-performance JavaScript and WebAssembly engine, written in C++. It powers Google Chrome and [Node.js](https://nodejs.org/).



V8 Architecture Overview





Step-by-Step Breakdown

Step 1: Scanner (Lexical Analysis / Tokenizing)

The scanner reads your raw JavaScript code character by character and breaks it into **tokens**.

Source: `const sum = 10 + 20;`

Tokens: `[const] [sum] [=] [10] [+] [20] [;]`

It also handles **lazy parsing** — meaning it skips parsing functions that aren't immediately needed to improve startup speed.



Step 2: Parser (Syntax Analysis)

The parser takes the tokens and builds an **Abstract Syntax Tree (AST)**.

V8 uses two parsers for performance:

Parser Type	Purpose
Pre-Parser (Lazy)	Quickly scans functions that aren't called immediately, skipping full analysis
Full Parser (Eager)	Fully parses code that is needed right away

AST for: `const sum = 10 + 20;`

```
VariableDeclaration (const)
  /      \
Identifier BinaryExpression (+)
 (sum)    /      \
          Literal  Literal
          (10)     (20)
```

If there's any **syntax error**, it gets caught here.



Step 3: Ignition (Interpreter)

Ignition is V8's **interpreter**. It takes the AST and converts it into **bytecode**.

AST → Ignition → Bytecode

Example Bytecode:

```
LdaSmi [10]      // Load small integer 10
Star r0          // Store in register r0
LdaSmi [20]      // Load small integer 20
Add r0           // Add r0 (10) + 20
Star r1          // Store result in r1
```

Why bytecode? Bytecode is smaller than machine code, uses less memory, and allows faster startup compared to compiling everything upfront.

Ignition also collects **feedback data** (type information, frequency of execution) while executing bytecode. This data is critical for the next step.

Step 4 : Execution + Profiling

Step 4: Execution + Profiling

As the bytecode runs, V8's internal **profiler** monitors the code and collects data:

What it Tracks	Why
How many times a function is called	To identify "hot" functions
Data types used in operations	To make type-based optimizations
Which branches are taken	To optimize control flow

If a piece of code is executed repeatedly (becomes "**hot**"), V8 sends it to TurboFan.



Step 5 : TurboFan (JIT)

Step 5: TurboFan (Optimizing JIT Compiler)

TurboFan is V8's **optimizing compiler**. It takes the hot bytecode along with the profiling data and compiles it into **highly optimized machine code**.

```
Ignition Bytecode + Profiling Data → TurboFan → Machine Code
```

Optimizations TurboFan performs:

Optimization	What it Does
Inline Caching	Remembers object property locations to avoid repeated lookups
Function Inlining	Replaces function calls with the function body itself
Dead Code Elimination	Removes code that never runs
Constant Folding	Calculates constant expressions at compile time (e.g., $2+3 \rightarrow 5$)
Loop Optimization	Optimizes loops for better performance
Escape Analysis	Avoids unnecessary memory allocations
Type Specialization	Generates code optimized for specific data types

Example of Type Specialization:

```
javascript
function add(a, b) {
  return a + b;
}

add(5, 10);    // TurboFan sees: always numbers
add(3, 7);     // Confirms: always numbers
// → Generates optimized machine code for integer addition
```

Step 6: De-optimization (Bailout)

If TurboFan's assumptions turn out to be wrong, V8 **throws away** the optimized machine code and falls back to Ignition's bytecode.

```
javascript

function add(a, b) {
  return a + b;
}

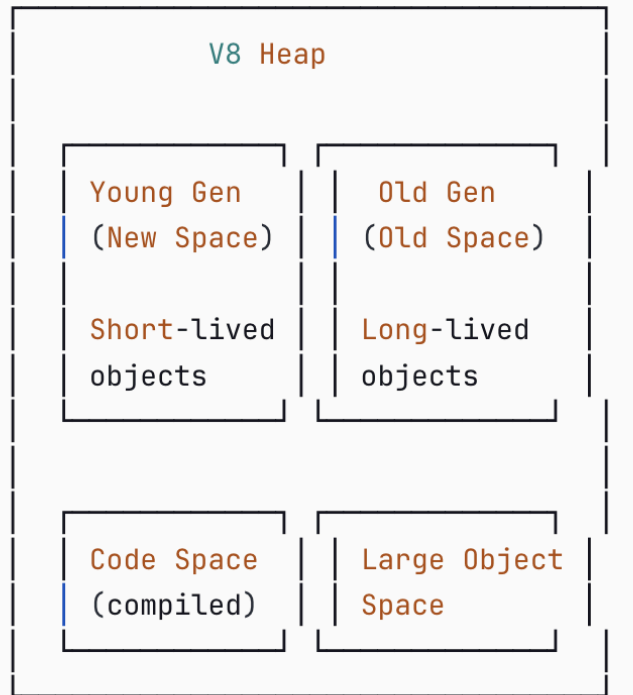
add(5, 10);           // ✅ Numbers → Optimized
add(3, 7);            // ✅ Numbers → Still optimized
add("hello", 5);      // ❌ String! → De-optimized, back to bytecode
...

```

Memory Management in V8

Memory Heap Structure

...



Step 7: Garbage Collection:

Garbage Collection

V8 uses two garbage collectors:

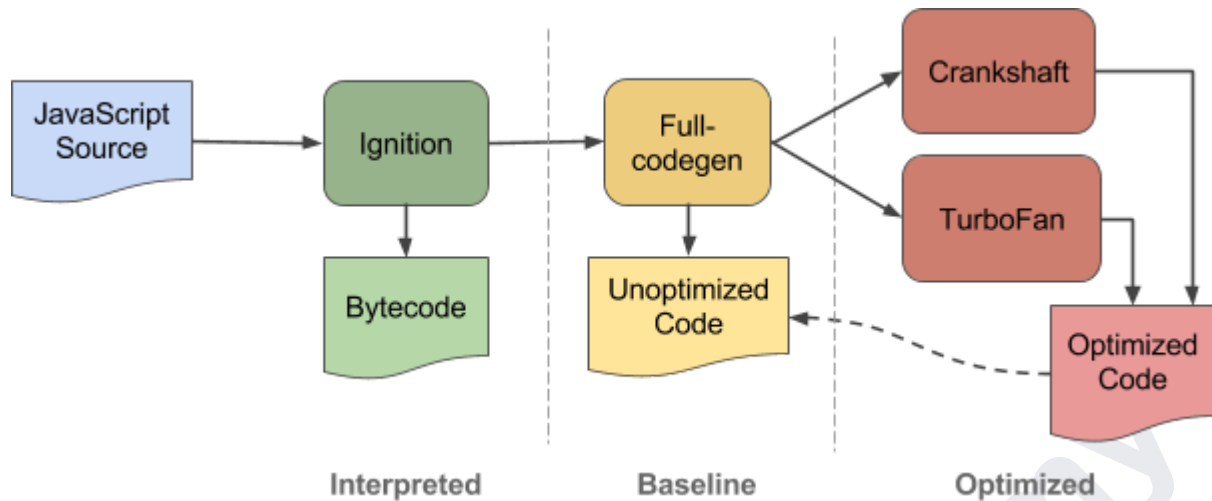
Collector	Area	How it Works
Scavenger (Minor GC)	Young Generation	Fast, frequent. Moves surviving objects to Old Gen
Mark-Compact (Major GC)	Old Generation	Slower, less frequent. Marks live objects, removes dead ones, compacts memory

Complete V8 Pipeline Summary

Stage	Component	Output	Purpose
1	Scanner	Tokens	Break code into meaningful pieces
2	Parser	AST	Build code structure
3	Ignition	Bytecode	Quick interpretation + profiling
4	Profiler	Feedback Data	Identify hot code + types
5	TurboFan	Machine Code	Maximum performance for hot code
6	De-optimization	Back to Bytecode	Fallback when assumptions fail
7	Garbage Collector	Freed Memory	Clean up unused objects

V8 takes your JavaScript, **parses** it into an AST, **interprets** it to bytecode via **Ignition** for fast startup, then **JIT compiles** hot code via **TurboFan** into optimized machine code, while managing memory through **generational garbage collection** and speeding up property access with **hidden classes** and **inline caching**.





Reference Article

- <https://dev.to/laxminarayana31/how-javascript-really-works-1p6i>
- <https://v8.dev/blog/ignition-interpreter>
- <https://medium.com/@manikmudholkar831995/the-v8-javascript-engine-d1434ca77c96>
- <https://v8.dev/>

ECMAScript Standard

What is ECMAScript?

ECMAScript (ES) is the **official standard/specification** that defines how JavaScript should work. Think of it as the **rulebook** that JavaScript follows.

It is maintained by **ECMA International** under the specification name **ECMA-262**, and developed by a committee called **TC39 (Technical Committee 39)**.

In simple terms:

- ECMAScript tells WHAT JavaScript should do
- JavaScript is HOW it's actually done in browsers and engines
- ES6 (ES2015) — The Game Changer

ES6 was the most important update in JavaScript history.



Category	Languages	How it works
AOT only	Go, C, C++, Rust	Compiled to machine code before running. Most errors caught at compile time
AOT + JIT	Java, C#	Compiled to bytecode (AOT), then JIT compiled to machine code at runtime
JIT only	JavaScript	Interpreted + JIT at runtime. Errors caught only at runtime
Interpreted only	Python (CPython)	Interpreted line by line. No JIT. All errors at runtime
AOT transpile + JIT	TypeScript	Type-checked & transpiled to JS (AOT), then JS runs with JIT

Node `--trace-opt` [file.js](#)

1. Simple hell
2. For loop



Chapter 02: Variables (var, let, const)

Variables in JavaScript: Comments, Identifiers, Keywords, Literals, var, let, const

What are Variables?

Variables are containers for storing data values. In JavaScript, we have three ways to declare variables:

- **var** → Function Scoped (old way, ES5)
 - **let** → Block Scoped (modern, ES6+)
 - **const** → Block Scoped + Cannot be reassigned (modern, ES6+)
-

Comments in JavaScript

Comments are lines of code that **will not be executed** by the JavaScript engine. They are used to explain the code, make it more readable, or temporarily disable code.

Single Line Comment

Use `//` to write a single line comment. Everything after `//` on that line is ignored.

```
// This is a comment. This code will not be executed
console.log("Hello");
```

Multi Line Comment

Use `/* */` or `/** */` to write multi-line comments. Everything between the opening and closing is ignored.

```
/**
 * This is multi line
 * Author : Pramod Dutta
 * Date : 14-Feb-2026
 */
```



```
/*  
 * This is multi line  
 * Author : Pramod Dutta  
 * Date : 14-Feb-2026  
 */
```

When to use Comments

- To explain **what** the code does
 - To add **author** and **date** information
 - To **temporarily disable** code during debugging
 - To write **documentation** for functions and modules
-

Identifier, Literal, and Operator

Every line of JavaScript code is made up of these basic building blocks. Let us understand each one.

```
var a = 10;
```

In the above line:

- **Identifier** → **a** (the name given to the container/variable)
- **Literal** → **10** (the actual value stored)
- **Operator** → **=** (assignment operator)
- **Keyword** → **var** (reserved word that tells JavaScript to create a variable)



Chapter 03: Identifiers, Literals & Operators

What is a Keyword?

A **keyword** is a reserved word in JavaScript that has a special meaning. You **cannot** use keywords as variable names.

Examples of Keywords: `var`, `let`, `const`, `function`, `if`, `else`, `for`, `while`, `return`, `class`, `import`, `export`, `new`, `this`, `typeof`, `null`, `undefined`, `true`, `false`

```
// var is a keyword, a is an identifier, 10 is a literal
var a = 10;
```

What is a Literal?

A **literal** is a fixed value that is directly written in the code. It represents the actual data.

Examples of Literals:

- **Number Literal** → `10`, `20`, `3.14`
- **String Literal** → `"Hello"`, `"Pramod"`
- **Boolean Literal** → `true`, `false`
- **Null Literal** → `null`
- **Undefined Literal** → `undefined`

```
var a = 10;      // 10 is a Number Literal
var name = "Hello"; // "Hello" is a String Literal
var flag = true;  // true is a Boolean Literal
```

What is an Identifier?

An **identifier** is the name given to a variable, function, class, or any other user-defined item. Think of it as the **name of the container** that holds the value.

```
var a = 10;
a = 20;
console.log(a);
// a is the Identifier (name of the container)
// 10 and 20 are the Literals (values)
```



Rules of Identifier

There are strict rules for naming identifiers in JavaScript:

1. Must start with a **letter**, **underscore _**, or **dollar sign \$**
2. Can contain letters, numbers, underscores, and dollar signs
3. **Cannot** start with a number
4. **Cannot** be a reserved keyword
5. **Cannot** contain spaces
6. **Cannot** contain special characters other than underscore and dollar sign
7. Identifiers are **case sensitive** (**NAME** and **name** are different)

Valid Identifier Examples

```
var name = "Hello";  
console.log(name);
```

```
var $name = "Hello";  
console.log($name);
```

```
var _name = "Hello";  
console.log(_name);
```

```
var name1 = "Hello";  
console.log(name1);
```

```
var name_1 = "Hello";  
console.log(name_1);
```

```
var name$1 = "Hello";  
console.log(name$1);
```

```
var name_1$1 = "Hello";  
console.log(name_1$1);
```

```
var name_1$1_2 = "Hello";  
console.log(name_1$1_2);
```

Case Sensitivity Example

```
var NAME = "pramod";  
var name1 = "dutta";  
console.log(NAME); // pramod  
console.log(name1); // dutta
```



// NAME and name1 are two completely different variables

Unicode Identifiers (Yes, JavaScript supports them!)

JavaScript allows Unicode characters in identifiers. So you can use characters from other languages too.

```
var π = 3.14;  
console.log(π); // 3.14
```

```
var नमस्ते = "Hi";  
console.log(नमस्ते); // Hi
```

```
var _ = "Hello";  
console.log(_); // Hello
```

These are **valid** but not recommended for production code. Stick to English letters for readability and team collaboration.



var (Function Scoped)

var is the oldest way to declare variables in JavaScript. It is **function scoped**, meaning it is accessible anywhere inside the function where it is declared. It does **NOT** respect block scope (like **if**, **for** blocks).

Example 1: Global Scope

```
var a = 10; // Global Scope  
console.log(a); // 10
```

Example 2: var inside a Function

```
// Definition of the function  
function printHello(){  
  console.log("Hello TheTestingAcademy!");  
  var a = 20; // Local Scope  
  console.log(a); // 20  
  
  if(true){  
    var a = 30;  
    console.log(a); // 30  
  }  
}
```



```
    console.log("F ->", a); // 30  
}
```

```
// Calling of the function  
printHello();
```

Why does `console.log("F ->", a)` print 30?

Because `var` is **function scoped**, not block scoped. The `var a = 30` inside the `if` block is the **same variable** as `var a = 20` in the function. It overwrites it. There is no separate block scope for `var`.

Example 3: var allows Re-declaration

```
// var also allows re-declaration:  
var a = 10;  
var a = 20; // No error, re-declaration is allowed  
console.log(a); // 20
```

Key Points about var

- `var` is **function scoped**
- Inside an `if` block, `var a = 30` overwrites the same `a` in the function scope
- `var` allows **re-declaration** in the same scope without any error
- `var` declarations are **hoisted** to the top of their scope

let (Block Scoped)

`let` was introduced in **ES6 (ES2015)**. It is **block scoped**, meaning it only exists inside the block `{ }` where it is declared. Each block gets its own separate copy of the variable.

Example 1: Global Scope

```
let b = 20; // Global Scope  
console.log(b); // 20
```

Example 2: let inside a Function

```
// Defination of the function  
function printHello(){  
    console.log("Hello TheTestingAcademy!");  
    let b = 30; // Local Scope
```



```
console.log(b); // 30

if(true){
  let b = 5;
  console.log(b); // 5
}

console.log("let ->", b); // 30
}

// Calling of the function
printHello();
```

Why does `console.log("let ->", b)` print 30?

JavaScript

```
// SCOPE REFERENCE: let
// let is block-scoped {}. A variable declared inside a block
// is distinct from one outside, even with the same name.
```

Example 3: let does NOT allow Re-declaration in the same scope

JavaScript

```
/**
 * Example 3: Re-declaration Check
 * let DOES NOT allow re-declaration in the same scope.
 */
let b = 30;
// let b = 40; // SyntaxError: Identifier 'b' has already been
// declared
```

Example 4: let allows Reassignment

JavaScript

```
/**
 * Example 4: Reassignment Check
 * let ALLOWS reassignment (mutable value).
 */
```



```
let a = 10;
a = 20;           // Valid reassignment
console.log(a);  // Output: 20
```

Example 5: Cannot mix var and let for the same name

```
JavaScript
/**
 * Example 5: Namespace Conflicts
 * Cannot mix 'var' and 'let' for the same identifier.
 */
var nn = "Pramod";
// let nn = "Pramod"; // Error: Identifier 'nn' has already been
// declared
```

Key Points about let

-

```
JavaScript
/**
 * TECHNICAL SUMMARY: let
 * - Scope: Block scoped (if, for, while, {})
 * - Shadowing: Each block gets a unique instance
 * - Re-declaration: Prohibited in same scope
 * - Reassignment: Permitted
 * - Hoisting: Hoisted to top of block but uninitialized (TDZ)
 */
```

const (Block Scoped + Constant)

```
JavaScript
/**
 * ES6 REFERENCE: const
 * Immutable reference. Must be initialized on declaration.
 */
const pi = 3.14;
```



```
// pi = 3.14159; // TypeError: Assignment to constant variable
```

Example 1: Declaring a constant

Key Points about const

-

JavaScript

```
/**  
 * TECHNICAL SUMMARY: const  
 * - Scope: Block scoped  
 * - Reassignment: Strictly Prohibited  
 * - Re-declaration: Strictly Prohibited  
 * - Initialization: Required at declaration  
 * - Mutation: Objects/Arrays remain mutable (reference is  
constant)  
 */
```

var vs let vs const: Quick Comparison

Feature	var	let	const
Scope	Function Scoped	Block Scoped	Block Scoped
Re-declaration	Allowed	Not Allowed	Not Allowed
Reassignment	Allowed	Allowed	Not Allowed
Hoisting	Hoisted + initialized as undefined	Hoisted but NOT initialized (TDZ)	Hoisted but NOT initialized (TDZ)



Must Initialize	No	No	Yes
Introduced In	ES1 (original)	ES6 (2015)	ES6 (2015)

var vs let: Scope Difference (Visual)

```
function example() {  
  // var → Function Scope  
  // The ENTIRE function is one scope for var  
  
  if (true) {  
    var x = 10; // Same scope as function  
    let y = 20; // Only inside this if block  
  }  
  
  console.log(x); // 10 (var is function scoped, still accessible)  
  console.log(y); // Error! (let is block scoped, not accessible here)  
}
```

When to Use What?

- Use **const** by default for all variables
- Use **let** when you know the value will change (counters, flags, reassignments)
- Avoid **var** in modern JavaScript (it can cause unexpected bugs due to function scoping)

In Simple Terms

const = "I will never change this value"

let = "I might change this value later"

var = "Old way, avoid using it"



Chapter 04: Operators

Operators in JavaScript

An operator is a symbol that performs an operation on values (called operands). In the expression $10 + 3$, the $+$ is the operator and 10 and 3 are the operands. JavaScript groups its operators into a few main types:

JavaScript

// CHEAT SHEET: Operators

```
const operators = {  
  arithmetic: ["+", "-", "*", "/", "%", "**"],  
  assignment: ["=", "+=", "-=", "*=", "/=", "%="],  
  comparison: ["==", "===", "!=", "!==", ">", "<", ">=", "<="],  
  logical: ["&&", "||", "!"],  
  string: ["+"],  
  ternary: ["condition ? val1 : val2"],  
  type: ["typeof"],  
  nullish: ["??"]  
};
```

Arithmetic Operators

Arithmetic operators perform basic math. Let us take two variables and try each one.

JavaScript

// MODULE: Arithmetic Operations

```
let a = 10, b = 3;  
console.log(a + b); // Addition: 13  
console.log(a - b); // Subtraction: 7  
console.log(a * b); // Multiplication: 30  
console.log(a / b); // Division: 3.33...  
console.log(a % b); // Modulus (Remainder): 1  
console.log(a ** b); // Exponentiation: 1000
```



Assignment Operators

The `=` operator assigns a value to a variable. JavaScript also gives you compound assignment operators, which do a calculation and assign the result back to the same variable in one short step.

JavaScript

```
// LOGIC: Conditionals (if/else)
/**
 * TRUTHY: non-empty string, non-zero num, {}, []
 * FALSY: "", 0, null, undefined, NaN, false
 */
function evaluate(val) {
  if (val) {
    return "Truthy - Block Executes";
  } else {
    return "Falsy - Block Skipped";
  }
}
```

Operator	Example	Same As
<code>+=</code>	<code>x += 10</code>	<code>x = x + 10</code>
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	<code>x *= 2</code>	<code>x = x * 2</code>
<code>/=</code>	<code>x /= 4</code>	<code>x = x / 4</code>
<code>%=</code>	<code>x %= 4</code>	<code>x = x % 4</code>

Comparison Operators

A comparison operator compares two values and **always returns a boolean** (true or false). These are the backbone of every `if` condition you will ever write. First understand the three kinds of "equals":

- `=` is the Assignment operator (puts a value into a variable)
- `==` is Loose comparison (checks the value only, ignores the data type)
- `===` is Strict comparison (checks the value AND the data type)



Think of it like comparing two people. `==` is a loose match ("both are human").
`===` is a strict match ("same religion, same language, same city"), everything has to match.

```
console.log(3 > 4); // false
```

```
console.log(3 < 4); // true
```

```
console.log(4 >= 4); // true (4 > 4 OR 4 === 4, an OR check)
```

```
console.log(3 <= 4); // true
```

```
console.log(5 == "5"); // true (loose: value matches, type ignored)
```

```
console.log(5 === "5"); // false (strict: number vs string, type differs)
```

```
console.log(5 != "5"); // false (loose not-equal: values match)
```

```
console.log(5 !== "5"); // true (strict not-equal: types differ)
```

Why does `5 == "5"` give true but `5 === "5"` give false?

Because `==` checks the value only, so the number 5 and the string "5" are treated as equal.
But `===` checks the value AND the data type, and a number is not the same type as a string, so it returns false.

Key Points about Comparison Operators

- Comparison operators always return true or false
- `==` checks value only (loose), `===` checks value and data type (strict)
- Always prefer `===` in real code to avoid type-coercion bugs
- `!=` is loose not-equal, `!==` is strict not-equal

Logical Operators

Logical operators combine or invert boolean values. There are three of them:

- `&&` (AND gate): true only when BOTH sides are true
- `||` (OR gate): true when AT LEAST ONE side is true
- `!` (NOT): reverses the value, true becomes false and false becomes true

```
let a = true;
```

```
let b = false;
```

```
console.log(a && b); // false (AND: both must be true)
```



```
console.log(a || b); // true (OR: one true is enough)
```

```
console.log(!a); // false (NOT: reverse of true)
```

A	B	A && B	A B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

String Operators

The `+` operator does double duty. With numbers it adds, but with strings it joins them together (this is called concatenation). The `+=` shortcut also works on strings.

```
let s = "Hi";
```

```
s += " Dev"; // same as s = s + " Dev"
```

```
console.log(s); // Hi Dev
```

When `+` is used with at least one string, JavaScript concatenates the values instead of adding them.

Ternary Operator

The ternary operator is a short one-line if-else. It checks a condition and returns one of two values.

Syntax: `condition ? valueIfTrue : valueIfFalse`

```
let age = 20;
```

```
let is_pramod_will_go_to_goa = age > 18 ? "Yes, let go goa!" : "No you are minor, Not going";
```

```
console.log(is_pramod_will_go_to_goa); // Yes, let go goa!
```

```
// Ternary = condition ? "true result" : "false result";
```

Here `age > 18` is true, so the value after the `?` is returned. If age were 15, the value after the `:` would be returned instead.



typeof Operator

The **typeof** operator tells you the data type of a value. It always returns a string like "number", "string", or "object".

```
console.log(typeof "hello"); // string

console.log(typeof 123);    // number (integers are numbers)

console.log(typeof 31.4);   // number (floats are also numbers)

console.log(typeof true);   // boolean

console.log(typeof undefined); // undefined

console.log(typeof null);   // object (a famous old JavaScript quirk)

console.log(typeof []);     // object (arrays are objects)

console.log(`${6 * 4}`);    // 24 (template literal, expression inside ${ })
```

Key Points about typeof

- **typeof** returns the type as a string
- Both integers and floats report as "number"
- **typeof null** returns "object", this is a long-standing bug kept for backward compatibility
- Arrays also report as "object"

Nullish Coalescing Operator (??)

The **??** operator gives you a default value only when the left side is null or undefined. It is very useful for handling test data and optional values.

```
let amul = null;

let val = amul ?? "nandani milk"; // amul is null, so val becomes "nandani milk"

let val2 = null ?? "default";    // "default" (?? returns the right side when left is null/undefined)

val = "which milk? -> " + val;

console.log(val); // which milk? -> nandani milk
```

Key Points about ??



- `??` returns the right-hand value only when the left is null or undefined
- Very useful in test data handling and default values
- It is a modern JavaScript / TypeScript feature

In Simple Terms

- Arithmetic does the math
- Comparison asks a yes/no question (returns true or false)
- Logical combines those yes/no answers
- Ternary is a one-line if-else
- `typeof` tells you the data type
- `??` provides a safe default when a value is missing

Chapter 05: Statements (if / else)

What is a Statement?

A statement is a single instruction that tells the computer to do something. A JavaScript program is just a list of statements that run one after another from top to bottom. The most common decision-making statement is the `if` statement, which runs a block of code only when a condition is true.

The if / else Statement

Use `if` to run code when a condition is true, and `else` to run different code when it is false. The condition inside the `( )` must result in a boolean (true or false).

```
let age = 20;

if (age > 18) {
    console.log("Yes, Goa");
} else {
    console.log("Go home!");
}
```

Here `age > 18` is true (20 is greater than 18), so "Yes, Goa" is printed. If age were 15, the `else` block would run instead and print "Go home!".



Key Points about if / else

- The condition inside () must be true or false
- The `if` block runs when the condition is true
- The `else` block runs when the condition is false
- `else` is optional, you can have an `if` on its own

The else if Ladder

When you have more than two choices, chain conditions with `else if`. JavaScript checks each condition from top to bottom and runs the **first** one that is true, then skips the rest.

```
let score = 78;
```

```
if (score >= 90) {  
    console.log("Grade: A — Excellent");  
}  
else if (score >= 80) {  
    console.log("Grade: B — Good");  
}  
else if (score >= 70) {  
    console.log("Grade: C — Average");  
}  
else if (score >= 60) {  
    console.log("Grade: D — Below Average");  
}  
else {  
    console.log("Grade: F — Fail");  
}
```

With `score = 78`, the first two checks are false, but `score >= 70` is true, so it prints `"Grade: C — Average"` and stops. Order matters here: always go from the highest range to the lowest.

Nested if (Real Live Example: User Roles)

You can put an `if` inside another `if`. This is called nesting. A real testing example: on an app like app.vwo.com a user must first be logged in, and then their role decides what they can do.



```
let isLoggedIn = true;

let userRole = "editor";

// app.vwo.com -> viewer, editor or admin ->

// viewer = limited view

// editor can edit and view

// admin can do all the things

if (isLoggedIn) {

  if (userRole === "admin") {

    console.log("admin can do all the things");

  } else if (userRole === "editor") {

    console.log("Welcome Editor — Edit access granted.");

  } else if (userRole === "viewer") {

    console.log("Welcome Viewer — Read-only access.");

  } else {

    console.log("No idea which role you are !");

  }

} else {

  console.log("You are not logged in!!!")

}
```

The outer **if** checks login first. Only if the user is logged in does the inner **else if** ladder check the role. Since **userRole** is **"editor"**, it prints **"Welcome Editor — Edit access granted."**.

if / else with API Status Codes

A very common testing use case: check the status code returned by an API and react to it.

```
let statusCode = 200; // APIs are working fine
```



```
if (statusCode == 200) {  
    console.log("Working fine!")  
} else if (statusCode == "404") {  
    console.log("not found!");  
} else {  
    console.log("Not mathcing status code!")  
}
```

Since `statusCode` is `200`, it prints `"Working fine!"`. Notice the second check uses `statusCode == "404"` (a string). The loose `==` would still match a number `404` to the string `"404"`, but in real code prefer `===` and compare against the number `404`.

■ Truthy and Falsy Values (Interview Favourite)

When a value is used inside an `if` condition, JavaScript converts it to true or false. Values that behave like true are called **truthy**, values that behave like false are called **falsy**.

//true

```
if ("hello") console.log("String is truthy"); // "hello" = truthy
```

```
if (42) console.log("Number is truthy");
```

```
if ({}) console.log("Empty object is truthy!");
```

```
if ([]) console.log("Empty array is truthy!");
```

// false results

```
if ( "") console.log("Won't print"); // "" -> falsy result
```

```
if (null) console.log("Won't print");
```

```
if (undefined) console.log("Won't print");
```

```
if (NaN) console.log("Won't print");
```

```
if (0) console.log("Won't print");
```

There are only a few falsy values in JavaScript. Everything else is truthy.



Falsy values (behave like false)	Truthy values (behave like true)
<code>false</code>	any non-empty string like <code>"hello"</code>
<code>0</code>	any non-zero number like <code>42</code>
<code>""</code> (empty string)	<code>"0"</code> and <code>"false"</code> (non-empty strings)
<code>null</code>	<code>{}</code> (empty object)
<code>undefined</code>	<code>[]</code> (empty array)
<code>NaN</code>	any function, date, etc.

Key Points about Truthy / Falsy

- Empty object `{}` and empty array `[]` are **truthy** (a common trap)
- The empty string `""`, `0`, `null`, `undefined`, and `NaN` are falsy
- The string `"0"` is truthy because it is a non-empty string

Logical Operators inside if

You can combine multiple conditions in one `if` using `&&` (AND), `||` (OR), and `!` (NOT).

```
let username = "Dev";

let password = "secure123";

let isAccountLocked = true;

if ((username === "Dev" && password === "secure123") && !isAccountLocked) {

    console.log("Allowed to enter");

} else {

    console.log("not allwed to enter");

}
```

The login is allowed only when the username matches AND the password matches AND the account is NOT locked (`!isAccountLocked`). Here `isAccountLocked` is true, so `!isAccountLocked` is false, and the whole condition fails, so it prints `"not allwed to enter"`.



if without Curly Braces

If you skip the `{ }`, only the **single next statement** belongs to the `if`. It still works for one line, but it is easy to make mistakes, so using braces is the safer habit.

```
let x = 10;
```

```
if (x > 5)
```

```
    console.log("x is big");
```

This prints `"x is big"` because `x > 5` is true. If you later add a second line under it, that second line would run every time (it is not part of the `if`), which is a classic bug.

Interview Questions (Common Traps)

```
if ([]) {
```

```
    console.log("True!")
```

```
}
```

```
// case 1
```

```
let response;
```

```
if (response) {
```

```
}
```

```
// case 2
```

```
if (response !== null) {
```

```
}
```

- `if ([])` prints `"True!"` because an empty array is truthy.
- Case 1: `response` is declared but has no value, so it is `undefined` (falsy), and the block is skipped.
- Case 2: `response !== null` is `true`, because `undefined` is not strictly equal to `null`, so this block would run. Watch out for this, `undefined` and `null` are different types.

In Simple Terms

- `if` runs code when the condition is true



- `else if` adds more conditions to check, in order
 - `else` is the fallback when nothing above matched
 - Empty `[]` and `{}` are truthy, while `""`, `0`, `null`, `undefined`, and `NaN` are falsy
-

Chapter 06: Switch Statements

The switch Statement

A `switch` statement is a cleaner way to compare one value against many possible options. Instead of writing a long `if / else if` ladder, you list each option as a `case`. It is perfect when you are checking a single variable against a list of fixed values.

```
// Switch

// 0 - Sunday, 1 - Monday, 2 - Tue.....

let day = 2;

switch (day) {

  case 0:

    console.log("Sunday — Rest Day");

    break;

  case 1:

    console.log("Monday — Sprint Planning");

    break;

  case 2:

    console.log("Tuesday — Development");

    break;

  case 3:

    console.log("Wednesday — Code Review");
```



```
        break;

    case 4:

        console.log("Thursday — Testing");

        break;

    case 5:

        console.log("Friday — Deployment & Retro");

        break;

    case 6:

        console.log("Saturday — Rest Day");

        break;

    default:

        console.log("Invalid day value");

}
```

`day` is 2, so JavaScript jumps to `case 2`, prints "`Tuesday – Development`", and the `break` stops it there. The `default` block runs only when no case matches.

Key Points about switch

- `switch` compares the value against each `case`
- `break` stops the switch once a match is handled
- `default` runs when no case matches (like the final `else`)
- `switch` uses strict comparison (`===`) to match cases

■ Why break Matters (Fall-Through)

If you forget `break`, JavaScript keeps running the next cases too, even though they did not match. This is called **fall-through**.

```
// Switch
```

```
// 0 - Sunday, 1 - Monday, 2 - Tue.....
```

```
let day = 2;
```



```
switch (day) {  
  
  case 0:  
  
    console.log("Sunday — Rest Day");  
  
  case 1:  
  
    console.log("Monday — Sprint Planning");  
  
  case 2:  
  
    console.log("Tuesday — Development");  
  
  case 3:  
  
    console.log("Wednesday — Code Review");  
  
  case 4:  
  
    console.log("Thursday — Testing");  
  
  case 5:  
  
    console.log("Friday — Deployment & Retro");  
  
  case 6:  
  
    console.log("Saturday — Rest Day");  
  
  default:  
  
    console.log("Invalid day value");  
  
}
```

With no **break**, matching **case 2** prints Tuesday **and then keeps going**, printing Wednesday, Thursday, Friday, Saturday, and the default line too. Always add **break** unless you deliberately want fall-through.

The default Case

default handles every value that none of the cases matched. It is the safety net.

// Switch



```
// 0 - Sunday, 1 - Monday, 2 - Tue.....
```

```
let day = 10;
```

```
switch (day) {
```

```
  case 0:
```

```
    console.log("Sunday — Rest Day");
```

```
    break;
```

```
  // ... cases 1 to 6 ...
```

```
  default:
```

```
    console.log("Invalid day value");
```

```
}
```

Here **day** is **10**, which matches no case, so only **"Invalid day value"** prints.

Real Example: API Response Codes

A very common testing use case is reacting to an API response code.

```
// You are working API Validation
```

```
// response Code - 200, 404, 401, 403.....404
```

```
let responseCode = 404;
```

```
switch (responseCode) {
```

```
  case 200:
```

```
    console.log("200 Ok");
```

```
    break;
```

```
  case 404:
```

```
    console.log("404 Not found!");
```

```
    break;
```

```
  default:
```



```
    console.log("Not status code match");  
  }  
}
```

`responseCode` is `404`, so it prints `"404 Not found!"`.

Grouping Cases (Shared Logic)

You can stack several `case` labels together so they all run the same code. This is a **good** use of fall-through.

```
let browser = "Edge";  
  
switch (browser) {  
  case "Chrome":  
  case "Edge":  
  case "Brave":  
  case "Opera":  
    console.log("Chromium Project!");  
    break;  
  case "Firefox":  
    console.log("Mozilla Project!");  
    break;  
  case "Safari":  
    console.log("Apple browser — uses JavaScriptCore engine");  
    break;  
  default:  
    console.log("Unknown browser — manual testing needed");  
}
```



Chrome, Edge, Brave, and Opera all fall into the same block, so "Edge" prints "Chromium Project!".

The switch(true) Pattern (Ranges)

Normal `switch` only matches exact values, so it cannot check ranges like `score >= 85`.

The trick is `switch (true)`: each case is a real condition, and the first one that evaluates to `true` wins.

```
let testScore = 85;

switch (true) {

  case (testScore >= 95):

    console.log("Outstanding — Top performer");

    break;

  case (testScore >= 85):

    console.log("Excellent — Above expectations");

    break;

  case (testScore >= 70):

    console.log("Good — Meets expectations");

    break;

  case (testScore >= 50):

    console.log("Needs Improvement");

    break;

  default:

    console.log("Unsatisfactory — Requires training");

}
```

`testScore` is 85. The first case (`>= 95`) is false, the second (`>= 85`) is true, so it prints "Excellent — Above expectations".



Interview Questions (Common Traps)

Missing break causes fall-through

```
let fruit = "banana";

switch (fruit) {

  case "apple":

    console.log("Apple selected");

  case "banana":

    console.log("Banana selected");

  case "cherry":

    console.log("Cherry selected");

  case "date":

    console.log("Date selected");

  default:

    console.log("Default reached");

}
```

Matching "banana" with no breaks prints **Banana selected**, **Cherry selected**, **Date selected**, and **Default reached**.

switch matches with === (strict), not ==

```
let value = "5";

switch (value) {

  case 5:

    console.log("Number 5 matched");

    break;

  case "5":
```



```
    console.log("String '5' matched");

    break;

}

// Output: "String '5' matched"

// switch uses ===, so "5" !== 5 (different types)

let status = 0;

switch (status) {

    case false:

        console.log("false matched");

        break;

    case 0:

        console.log("0 matched");

        break;

}

// Output: "0 matched" (0 === 0, NOT 0 === false)
```

Because `switch` uses strict `===`, the string `"5"` does not match the number `5`, and `0` does not match `false`.

In Simple Terms

- `switch` is a clean replacement for a long `if / else if` chain on one value
 - `break` stops it, without `break` it falls through to the next cases
 - `default` is the fallback for unmatched values
 - Group cases to share logic, use `switch (true)` for ranges
 - Matching is strict (`===`), so type matters
-



Chapter 07: Loops

Why Do We Need Loops?

A loop lets you repeat a block of code many times without writing it again and again. Imagine printing numbers 1 to 10 by hand:

```
console.log(1);
```

```
console.log(2);
```

```
console.log(3);
```

```
console.log(4);
```

```
console.log(5);
```

```
console.log("...");
```

```
console.log(10);
```

That is a lot of repetition. A loop does the same job in three lines. JavaScript gives you three main loops: `for`, `while`, and `do...while`.

Increment and Decrement Operators

Before loops, you must understand `++` and `--`. They add or subtract 1 from a variable. They come in two flavours, and the difference matters.

```
// ++ -> increase the value by 1
```

```
// -- -> decrease the value by 1
```

```
// Pre-increment (++a): increase FIRST, then use the value
```

```
let a = 10;
```

```
let b = ++a; // a becomes 11, then b = 11
```

```
console.log(a); // 11
```

```
console.log(b); // 11
```

```
// Post-increment (a++): use the value FIRST, then increase
```

```
let a = 10;
```



```
let b = a++; // b = 10 (old value), then a becomes 11
```

```
console.log(b); // 10
```

```
console.log(a); // 11
```

- **Pre (++a)**: increment happens before the value is used
- **Post (a++)**: the current value is used first, then it increments

This is the classic Expression Result Table (ERT) idea: track the value of `a` line by line to predict the output.

```
let a = 10;
```

```
let r2 = --a; // pre-decrement: a becomes 9, then r2 = 9
```

```
console.log(r2); // 9
```

The for Loop

The `for` loop is the most used loop. It packs three parts into one line: **Initialization**, **Condition**, and **Update** (remember it as `for (I ; C ; U)`).

```
// for(I;C;U)
```

```
for (let i = 0; i < 5; ++i) {
```

```
    console.log(i);
```

```
}
```

- **Initialization** `let i = 0`: runs once at the start
- **Condition** `i < 5`: checked before every round, the loop runs while it is true
- **Update** `++i` or `i++`: runs after every round

This prints `0 1 2 3 4`. In a `for` loop, using `++i` or `i++` in the update section gives the same result.

```
for (let somya = 0; somya < 10; somya++) {
```

```
    console.log(somya);
```

```
}
```

```
// 0 to 9
```



The counter starts at 0 and the condition is `somya < 10`, so it prints 0 through 9, which is 10 times. If you want 0 to 10 (11 times), use `<= 10`.

Watch out: a condition that is false at the start never runs

```
for (let i = 0; i > 10;) {  
  
    console.log("Hello");  
  
}
```

`i` starts at 0, and `0 > 10` is already false, so this loop body never runs even once.

Key Points about the for Loop

- Structure is `for (init ; condition ; update)`
- `i < 5` runs 5 times (0 to 4), `i <= 5` runs 6 times (0 to 5)
- If the condition is false at the start, the body runs zero times

The while Loop

The `while` loop is the sister of the `for` loop. It also needs Initialization, Condition, and Update, but they are written separately. Use it when you do not know in advance how many times you will loop.

```
// While Loop - Sister (For)
```

```
// I,C,u
```

```
let attempts = 0; // Init
```

```
while (attempts < 3) {
```

```
    console.log("Attempt", attempts);
```

```
    attempts++;
```

```
}
```

The counter `attempts` starts at 0, the loop runs while `attempts < 3`, and `attempts++` moves it forward. This prints Attempt 0, Attempt 1, Attempt 2.

```
let modi = 1;
```

```
while (modi <= 15) { // 1 to 15
```



```
console.log("Modi will do 15+ years");

modi++;

}
```

This runs 15 times (from 1 to 15). Never forget the update line (`modi++`), or the condition stays true forever and you get an infinite loop.

The do...while Loop

A `do...while` loop runs the body **first**, and checks the condition **after**. This guarantees the body runs at least once, even if the condition is false.

```
let a = 10;

do {

    console.log(a);

    a++;

} while (a < 10);
```

Here `a` is 10, so `a < 10` is false. But because `do...while` checks the condition **after** the first run, it still prints 10 once before stopping.

```
let number = 0;

do {

    console.log(number);

    // Code that we need to execute

    number++;

} while (number < 10);
```

This prints 0 through 9.

Key difference: a `while` loop may run zero times, but a `do...while` loop always runs at least once.

In Simple Terms



- Loops repeat code so you do not have to
 - `++` and `--` change a value by 1 (pre uses the new value, post uses the old value first)
 - Use `for` when you know the number of rounds
 - Use `while` when you loop until a condition changes
 - Use `do...while` when the body must run at least once
 - Always update the counter, or you create an infinite loop
-

Chapter 08: Arrays

What is an Array?

An array is a single container that holds a list of values in order. Instead of making many separate variables, you store them all in one variable and reach each item by its **index**. In JavaScript, indexes start at `0`.

```
// Array
```

```
let fruits = []; // Empty []
```

```
let fruits_fresh = ["apple", "banana", "cheery"];
```

```
// 3 items, index - 0,1,2
```

```
let arr = [10, 20, 30, 40]; // 0-3: 4 items
```

```
console.log(arr.length); // 4
```

```
console.log(arr[0]); // 10
```

```
console.log(arr[3]); // 40
```

```
console.log(arr[4]); // undefined (no item at index 4)
```

```
let testResults = ["pass", "fail", "pass", "skip"];
```

```
let mixed = [1, "hello", true, null]; // JS arrays can hold any type.
```

- `arr.length` is the number of items (`length` is a property, not a function, so no `()`)
- `arr[0]` is the first item, `arr[3]` is the last item here
- Reading an index that does not exist gives `undefined`
- A JavaScript array can mix numbers, strings, booleans, and null together



■ Creating Arrays

There are several ways to create an array. The **array literal** (square brackets) is the preferred one.

```
// Array literal (preferred)
```

```
let browsers = ["Chrome", "Firefox", "Safari"];
```

```
// Array constructor
```

```
let scores = new Array(3);    // creates [empty x 3]
```

```
let scores2 = new Array(1, 2, 3); // creates [1, 2, 3]
```

```
let numbers = new Array(100, 200, 300, 400); // 0-3: 4 items
```

```
console.log(numbers);
```

```
// Array.of() always builds from the given values
```

```
let test = Array.of(10, 20, 30, 40, 50); // 0-4: 5 items
```

```
console.log(test);
```

```
console.log(test[0]); // 10
```

```
// Array.from() builds an array from something iterable
```

```
let chars = Array.from("hello"); // ["h", "e", "l", "l", "o"]
```

Note the trap with `new Array(3)`: a single number makes an empty array of that **length**, not `[3]`. `Array.of(3)` would make `[3]`.

■ Accessing and Modifying

Read items by index, and use `.at()` to count from the end with negative numbers.

```
let statuses = ["pass", "fail", "skip"];
```

```
console.log(statuses[0]); // pass
```

```
console.log(statuses[2]); // skip
```

```
console.log(statuses.at(-1)); // skip (last element)
```

```
console.log(statuses.at(-2)); // fail
```



```
console.log(statuses.at(-3)); // pass

console.log(statuses.at(-4)); // undefined

// Modify an item by assigning to its index

statuses[1] = "blocked";

console.log(statuses); // ["pass", "blocked", "skip"]

// Length

console.log(statuses.length); // 3

.at(-1) is the clean modern way to get the last item.
```

Adding and Removing Items

These methods change (mutate) the array.

```
let arr = [1, 2, 3];

// Add to the END

arr.push(4);    // [1, 2, 3, 4]

// Remove from the END

arr.pop();      // [1, 2, 3]

arr.push(5, 6); // [1, 2, 3, 5, 6]

// Add to the BEGINNING

arr.unshift(0); // [0, 1, 2, 3, 5, 6]

// Remove from the BEGINNING

arr.shift();    // [1, 2, 3, 5, 6]

// splice(start, deleteCount, ...itemsToAdd)

arr.splice(2, 1);    // removes 1 item at index 2

arr.splice(2, 0, 99); // inserts 99 at index 2, removes nothing
```



```
arr.splice(1, 2, 10, 20); // removes 2 items from index 1, inserts 10 and 20
```

Method	What it does	Which end
<code>push</code>	adds item(s)	end
<code>pop</code>	removes item	end
<code>unshift</code>	adds item(s)	beginning
<code>shift</code>	removes item	beginning
<code>splice</code>	remove and/or insert anywhere	any position

Real Example: Browser List

```
let browser = ['chrome', 'firefox', 'safari', 'opera', 'edge'];
```

```
console.log(browser.length); // 5
```

```
console.log(browser);
```

```
browser.pop(); // removes 'edge'
```

```
console.log(browser);
```

```
let removed = browser.shift(); // removes 'chrome'
```

```
console.log(browser);
```

```
console.log(removed); // 'chrome'
```

```
for (let i = 0; i < browser.length; i++) {
```

```
    console.log(browser[i]);
```

```
    if (browser[i] === "opera") {
```

```
        console.log("Opera is removed from the selenium!");
```

```
    }
```

```
}
```

`shift()` returns the item it removed, so `removed` holds `'chrome'`. The `for` loop then walks the remaining browsers and reacts when it finds `"opera"`.



■ Searching in Arrays

```
let results = ["pass", "fail", "pass", "error", "fail"];

// indexOf — first matching index, or -1 if not found

results.indexOf("fail");    // 1

results.indexOf("skip");    // -1

// lastIndexOf — searches from the end

results.lastIndexOf("fail"); // 4

// includes — returns true or false

results.includes("error");  // true

results.includes("skip");   // false

// find — returns the first element that matches a test

let nums = [10, 25, 30, 45];

nums.find(x => x > 20);      // 25

// findIndex — index of the first match

nums.findIndex(n => n > 20); // 1

// findLast / findLastIndex — search from the end

nums.findLast(n => n > 20);   // 45

nums.findLastIndex(n => n > 20); // 3

- Use includes for a simple yes/no
- Use indexOf when you need the position
- Use find when you want the actual element that matches a condition
```

■ Iterating (Looping) Over Arrays

There are several ways to walk through an array. Pick the one that reads best.

```
let tests = ["login", "checkout", "search"];

// Classic for loop (full control over the index)
```



```
for (let i = 0; i < tests.length; i++) {  
    console.log(tests[i]);  
}  
  
// for...of (cleanest way to get the VALUES)  
  
for (let test of tests) {  
    console.log(test); // value  
}  
  
// forEach (runs a function for each item, returns nothing)  
tests.forEach((test, index) => {  
    console.log(`${index}: ${test}`);  
});  
  
// entries() — gives both index and value  
for (let [i, test] of tests.entries()) {  
    console.log(i, test);  
}  
  
// for...in gives the INDEX (as a string key)  
  
let students = ["methis", "senthil", "ajay", "rahul"];  
  
for (let student in students) {  
    console.log(student, " -> ", students[student]); // student is the index  
}
```

Important difference: **for...of** gives you the **values**, while **for...in** gives you the **indexes** (keys). For arrays, prefer **for...of** or **forEach**.



■ Transforming Arrays (map, filter, reduce)

These are the most important array methods for testing and data work. **map** and **filter** return a **new** array and do not change the original.

```
let scores = [45, 82, 91, 60, 73];
```

```
// map — transforms every element, returns a NEW array
```

```
let grades = scores.map(s => s > 70 ? "Pass" : "Fail");
```

```
console.log(grades); // ["Fail", "Pass", "Pass", "Fail", "Pass"]
```

```
// filter — keeps only elements that pass a test
```

```
let passing = scores.filter(s => s >= 70);
```

```
console.log(passing); // [82, 91, 73]
```

```
// reduce — accumulates everything into a single value
```

```
let total = scores.reduce((sum, s) => sum + s, 0); // 351
```

```
console.log(total);
```

```
// flat — flattens nested arrays into one level
```

```
let nested = [[1, 2], [3, 4], [5]];
```

```
console.log(nested.flat()); // [1, 2, 3, 4, 5]
```

- **map**: same number of items, each one transformed
- **filter**: fewer items, only those that pass the test
- **reduce**: one final value (a sum, a count, an object, etc.). The **0** is the starting value

■ Sorting Arrays

sort() sorts alphabetically by default, which is a famous trap for numbers.

```
let fruits = ["banana", "apple", "cherry"];
```

```
fruits.sort();
```

```
console.log(fruits); // ["apple", "banana", "cherry"] (alphabetical)
```

```
let nums = [10, 1, 21, 2];
```



```
nums.sort();

console.log(nums); // [1, 10, 2, 21] <- WRONG (compares as strings!)

nums.sort((a, b) => a - b); // Ascending

console.log(nums); // [1, 2, 10, 21]

nums.sort((a, b) => b - a); // Descending

console.log(nums); // [21, 10, 2, 1]
```

For numbers you must pass a compare function: `(a, b) => a - b` for ascending, `(a, b) => b - a` for descending.

Slicing and Combining

`slice` copies a part of an array without changing the original. `concat` and the spread `...` join arrays.

```
let arr = [1, 2, 3, 4, 5];

// slice(start, end) — returns a new array, from start up to end-1

arr.slice(1, 3); // [2, 3]

arr.slice(2); // [3, 4, 5] (no end = go to the end)

arr.slice(-2); // [4, 5] (negative counts from the end)

let a = [1, 2];

let b = [3, 4];

// concat — joins arrays into a new one

let c = a.concat(b);

console.log(c); // [1, 2, 3, 4]

// spread (modern way to combine)

let d = [...a, ...b];

console.log(d); // [1, 2, 3, 4]

// join — turns an array into a string
```



```
let s = ["pass", "fail", "skip"].join(" | ");
```

```
console.log(s); // "pass | fail | skip"
```

Remember: `slice` does NOT mutate, but `splice` (from earlier) DOES mutate. Same letters, very different behaviour.

■ Checking Arrays

```
// Check if something IS an array
```

```
let result = Array.isArray([1, 2, 3]);
```

```
let result1 = Array.isArray("a");
```

```
console.log(result); // true
```

```
console.log(result1); // false
```

```
// every — ALL items must pass the test
```

```
[80, 90, 85].every(s => s >= 70); // true
```

```
[80, 60, 85].every(s => s >= 70); // false
```

```
// some — AT LEAST ONE item must pass the test
```

```
[80, 60, 85].some(s => s < 70); // true
```

```
[80, 90, 85].some(s => s < 70); // false
```

- `every` is like an AND across all items
- `some` is like an OR across all items

■ Copying Arrays (Shallow vs Reference)

This is a very common bug. Assigning one array to another with `=` does **not** copy it, both names point to the same array.

```
let arr = [1, 2, 3];
```

```
let copy = arr; // NOT a copy, same array
```

```
copy.push(4);
```

```
console.log(arr.length); // 4 (the original changed too!)
```



```
console.log(copy);    // [1, 2, 3, 4]
```

To make a real (shallow) copy, use spread, `slice`, `Array.from`, or `concat`.

```
let original = [1, 2, 3];
```

```
let copy1 = [...original];    // spread
```

```
let copy2 = original.slice();  // slice
```

```
let copy3 = Array.from(original); // from
```

```
let copy4 = original.concat(); // concat
```

```
copy1.push(99);
```

```
console.log(original); // [1, 2, 3] (untouched)
```

```
console.log(copy1);    // [1, 2, 3, 99]
```

Key Point: `let copy = arr` shares the same array (a reference). Use `[...arr]` or `arr.slice()` when you want an independent copy.



Array Destructuring

Destructuring lets you pull items out of an array into separate variables in one line. The `...rest` syntax grabs everything that is left over.

```
let [first, second, ...third] = [10, 20, 30, 40, 50];
```

```
console.log(first); // 10
```

```
console.log(second); // 20
```

```
console.log(third); // [30, 40, 50]
```

`first` and `second` take the first two items, and `...third` collects the rest into a new array.

In Simple Terms

- An array is an ordered list, indexes start at `0`
- `push` / `pop` work at the end, `shift` / `unshift` at the beginning, `splice` anywhere
- `map` transforms, `filter` keeps matches, `reduce` collapses to one value
- `sort` needs `(a, b) => a - b` for numbers



- `slice` copies (safe), `splice` mutates (changes the original)
 - `let b = a` is NOT a copy, use `[...a]` to truly copy
-

Chapter 09: Functions

Why Do We Need Functions?

A function is a reusable block of code that does one job. You write it once and call it as many times as you like. Without functions you repeat the same logic again and again.

// Without functions — repeated logic

```
let score1 = 85;
```

```
let result1 = score1 >= 70 ? "pass" : "fail";
```

```
console.log(result1);
```

```
let score2 = 45;
```

```
let result2 = score2 >= 70 ? "pass" : "fail";
```

```
console.log(result2);
```

// With a function — write once, reuse

```
function getResult(score) {
```

```
    return score >= 70 ? "pass" : "fail";
```

```
}
```

```
getResult(85); // "pass"
```

```
getResult(45); // "fail"
```

A function keeps your code short, readable, and easy to fix in one place.

The 4 Types of Functions

Every function is a combination of two questions: does it take **arguments**, and does it **return** a value? That gives four types.

Type 1: No argument, no return



```
function greet() {  
    console.log("Hi");  
}  
  
greet();  
  
let a = greet(); // a is undefined, because greet() returns nothing
```

Type 2: With argument, no return

```
function greetByName(name) { // parameter  
    console.log("Hi", name);  
}  
  
greetByName("Pramod"); // argument  
  
greetByName("Dipak");
```

A **parameter** is the name in the definition (**name**). An **argument** is the actual value you pass in ("**Pramod**").

Type 3: No argument, with return

```
function sayHello() {  
    console.log('Hi');  
    return "Hello";  
}  
  
let relative = sayHello();  
  
console.log(relative); // "Hello"
```

Type 4: With argument, with return (the most useful)

```
function sumofTwoNumbers(a, b) {  
    return a + b;  
}
```



```
let c = sumofTwoNumbers(4, 5); // 9
```

```
let c2 = sumofTwoNumbers(10, 15); // 25
```

`return` sends a value back to whoever called the function. A function with no `return` gives back `undefined`.

Function Expression

A function can be stored in a variable. This is called a function expression.

```
// Normal function declaration
```

```
function greet1(name1) {  
    return `Hello, ${name1}!`;  
}
```

```
// Function as an expression (stored in a const)
```

```
const greet2 = function (name1) {  
    return `Hello, ${name1}!`;  
};  
  
console.log(greet1("Bob")); // Hello, Bob!  
  
console.log(greet2("Bob")); // Hello, Bob!
```

Arrow Functions (ES6)

Arrow functions are a shorter way to write function expressions. To convert a normal function: remove the word `function`, and for a single-line body remove `return` and the `{ }`, then add `=>`.

```
const greet1 = (name2) => `Hello, ${name2}!`;  
  
console.log(greet1("Pramod")); // Hello, Pramod!  
  
// Single expression — return is automatic  
  
const doubleIt = n => n * 2;  
  
console.log(doubleIt(10)); // 20
```



```
// No parameters — empty parentheses are required

const getEnv = () => "staging";

console.log(getEnv()); // staging

// Multi-line — you need curly braces AND an explicit return

const getResult = (score) => {

  if (score >= 70) return "pass";

  return "fail";

};
```

The same logic can be written three ways, they all work the same:

```
function validateStatusCode(status) {

  if (status >= 200 && status <= 300) console.log("Request is fine!");

}

const validateStatusCode_Exp = function (status) {

  if (status >= 200 && status <= 300) console.log("Request is fine!");

};

const validateStatusCode_Arrow = (status) => {

  if (status >= 200 && status <= 300) console.log("Request is fine!");

};
```

IIFE (Immediately Invoked Function Expression)

An IIFE is a function that runs the moment it is defined. You do not call it separately, the `()` at the end calls it right away. It is useful for one-time setup code.

```
// IIFE

(function () {

  console.log("Hi")

})
```



```
})();  
  
(() => {  
    console.log("Setup complete");  
})();
```

Default, Rest, and Spread Parameters

Default parameters give a value when the caller does not pass one.

```
function retry(testName, maxRetries = 3, delay = 1000) {  
    console.log(`Retrying ${testName} up to ${maxRetries} times, ${delay}ms apart`);  
}  
  
retry("Login");           // uses 3 and 1000  
  
retry("Checkout", 5);     // uses 5 and 1000  
  
retry("API Test", 2, 500); // uses 2 and 500
```

Rest parameters (`...`) collect many arguments into a single array.

```
function logResults(suiteName, ...results) {  
    console.log(`Suite: ${suiteName}`);  
    console.log(`Results: ${results.join(", ")}`);  
}  
  
logResults("Auth Suite", "pass", "fail", "pass", "skip");  
  
// Results: pass, fail, pass, skip
```

Spread (`...`) does the opposite: it unpacks an array into individual arguments.

```
function add(a, b, c) {  
    return a + b + c;  
}
```



```
let num = [1, 2, 3];

add(...num); // 6

function hasError(...codes) {

    return codes.some(c => c >= 400);

}

let responseCodes = [200, 201, 404];

hasError(...responseCodes); // true
```

Return Values

return ends the function and sends a value back. A function can have several **return** lines, and the first one that runs wins.

```
function getStatus(code) {

    if (code >= 200 && code < 300) return "success";

    if (code >= 400 && code < 500) return "client error";

    if (code >= 500) return "server error";

}

getStatus(200); // "success"

getStatus(404); // "client error"

getStatus(500); // "server error"

// No return statement → the function gives back undefined

function logTest(name) {

    console.log(`Running: ${name}`);

}
```

To return more than one value, wrap them in an array or an object.



Hoisting

Function declarations are hoisted, meaning you can call them before they appear in the code. **Function expressions** and **arrow functions** are not.

```
greet("Alice"); // works — declarations are hoisted
```

```
function greet(name) {  
    return `Hello, ${name}!`;  
}
```

```
sayHi("Bob"); // TypeError: sayHi is not a function
```

```
const sayHi = function (name) {  
    return `Hi, ${name}!`;  
};
```

Scope in Functions

Scope decides where a variable can be seen. A variable declared inside a function is **local** and cannot be used outside it. A variable outside all functions is **global** and can be used anywhere.

```
let env = "staging"; // global scope
```

```
function setupConfig() {  
    let timeout = 3000; // local scope  
  
    console.log(env); // can access global  
  
    console.log(timeout); // can access local  
}
```

```
setupConfig();
```

```
console.log(env);
```

```
console.log(timeout); // ReferenceError — not accessible outside
```

An inner function can read the outer function's variables, but the outer function cannot read the inner one's.



Closures

A closure is a function that remembers the variables from the place where it was created, even after that outer function has finished. This is one of the most powerful ideas in JavaScript.

```
function outer() {  
  
    let message = "Hello";  
  
    console.log("Outer called!");  
  
    function inner() {  
  
        console.log(message); // inner still remembers message  
  
    }  
  
    return inner;  
  
}  
  
let fn_inner = outer();  
  
fn_inner(); // "Hello"
```

A closure is often used to keep a private value that only certain functions can change, like a counter.

```
function makeCounter(start = 0) {  
  
    let count = start; // this variable is "closed over" (private)  
  
    return {  
  
        increment() { count++; },  
  
        decrement() { count--; },  
  
        get() { return count; }  
  
    };  
  
}  
  
let counter = makeCounter(0);  
  
counter.increment();
```



```
counter.increment();

counter.increment();

console.log(counter.get()); // 3

counter.decrement();

console.log(counter.get()); // 2
```

A real testing example, a retry tracker that remembers how many attempts happened:

```
function makeRetryTracker(max) {

  let attempts = 0;

  function tryAgain(testName) {

    attempts++;

    if (attempts > max) {

      return `${testName} exceeded max retries (${max})`;

    }

    return `Attempt ${attempts}/${max} for ${testName}`;

  }

  return tryAgain;

}

let retry = makeRetryTracker(3);

console.log(retry("Login")); // Attempt 1/3 for Login

console.log(retry("Login")); // Attempt 2/3 for Login

console.log(retry("Login")); // Attempt 3/3 for Login

console.log(retry("Login")); // Login exceeded max retries (3)
```



Higher-Order Functions

A higher-order function is a function that takes another function as an argument, or returns a function. This is what makes `map`, `filter`, and callbacks possible.

```
function runWithLogging(testfn, testName) {  
  
    console.log(`Starting: ${testName}`);  
  
    let result = testfn();  
  
    console.log(`Finished: ${testName} → ${result}`);  
  
    return result;  
  
}  
  
function loginTest() { return "pass"; }  
  
function loginTestFAILED() { return "fail"; }  
  
runWithLogging(loginTest, "Login Test");  
  
runWithLogging(loginTestFAILED, "Dashboard Failed Test");
```

Callback Functions

A callback is a function passed as an argument to another function, to be called later (when some work is done).

```
function runTest(testName, callback) {  
  
    let result = "pass";  
  
    // ... do the test work ...  
  
    callback(testName, result);  
  
}  
  
function onComplete(name, result) {  
  
    console.log(`${name} finished with: ${result}`);  
  
}  
  
runTest("loginTest", onComplete); // loginTest finished with: pass
```



```
function pramod_doing_work(worker, callback) {  
    console.log("Started the class PW");  
  
    let work = worker;  
  
    console.log("Finished the class PW");  
  
    callback();  
}  
  
function callWife() {  
    console.log("Call wife when done");  
}  
  
pramod_doing_work('PW class', callWife);
```

Pure Functions

A pure function always returns the same output for the same input and has no side effects (it does not depend on or change anything outside itself). Pure functions are easy to test.

// Pure — predictable, no outside dependency

```
function calculatePassRate(total, passed) {  
    return ((passed / total) * 100).toFixed(2);  
}
```

```
console.log(calculatePassRate(10, 7)); // "70.00" every time
```

// Impure — depends on an external variable

```
function isPassing(score) {  
    return score >= threshold; // threshold lives outside  
}
```

```
let threshold = 70;
```

```
console.log(isPassing(70)); // true
```



```
threshold = 50;
```

```
console.log(isPassing(70)); // true — result changed because threshold changed
```

In Simple Terms

- A function is reusable code, write once and call many times
- Functions come in 4 types based on argument and return
- Arrow functions are a shorter form, and expressions store a function in a variable
- Default, rest (`...args`), and spread (`...arr`) control how arguments are passed
- Declarations are hoisted, expressions and arrows are not
- A closure remembers the variables from where it was created
- Higher-order functions take or return functions, callbacks run "later"
- Pure functions are predictable and the easiest to test

Chapter 10: Strings

What is a String?

A string is text: anything written inside quotes. You can use single quotes, double quotes, or backticks. Backticks (called template literals) are special because they allow expressions inside `${ }` and can span multiple lines.

```
let url = "https://app.vwo.com";
```

```
let status = 'pass';
```

```
let message = `Test completed in ${320}ms`; // template literal
```

```
// Template literals allow expressions and multiline text
```

```
let name1 = "Alice";
```

```
let msg = `Hello, ${name1}! 2 + 2 = ${2 + 2}`;
```

```
console.log(msg); // Hello, Alice! 2 + 2 = 4
```

```
let report = `
```

```
  Test: Login
```

```
  Status: Pass
```

```
  Duration: 320ms
```



```
`;
```

```
// String() converts other types into a string
```

```
console.log(String(200)); // "200"
```

```
String(true); // "true"
```

```
String(null); // "null"
```

```
String([1, 2]); // "1,2"
```

Use backticks whenever you want to drop a variable or a calculation into the middle of text.

String Properties and Access

```
let str = "Hello, World!";
```

```
// length is the number of characters
```

```
console.log(str.length); // 13
```

```
// Access a character by index (starts at 0)
```

```
console.log(str[0]); // "H"
```

```
console.log(str[7]); // "W"
```

```
console.log(str.at(-1)); // "!" (last character)
```

```
console.log(str.at(-6)); // "o"
```

```
// charAt() and charCodeAt()
```

```
str.charAt(0); // "H"
```

```
str.charCodeAt(0); // 72 (the Unicode number for "H")
```

`.at(-1)` is the clean way to get the last character.

Searching and Checking

```
let url = "https://staging.vwo.com/api/login?retry=true";
```

```
// includes() — is the text present? returns true/false
```



```
url.includes("staging"); // true

url.includes("production"); // false

// startsWith / endsWith

url.startsWith("https"); // true

url.startsWith("http://"); // false

url.endsWith("true"); // true

// indexOf / lastIndexOf — position of the text, or -1 if missing

console.log(url.indexOf("a")); // first "a"

console.log(url.lastIndexOf("a")); // last "a"

console.log(url.indexOf("nothere")); // -1

// search() accepts a regular expression and returns the index

console.log(url.search(/login/));

url.search(/\d+/); // finds the first digit
```

- Use `includes` for a simple yes/no
- Use `startsWith` / `endsWith` to validate URLs or file names
- Use `indexOf` when you need the exact position

■ Extracting Substrings

`slice` and `substring` both pull out a part of a string. `slice` supports negative indexes (counting from the end), `substring` does not.

```
let str = "Login_Test_Pass_001";

// slice(start, end) — end is not included, negatives allowed

console.log(str.slice(0, 5)); // "Login"

console.log(str.slice(11)); // "Pass_001"

console.log(str.slice(-3)); // "001" (last 3 characters)

let testNumber = str.slice(-3);
```



```
// substring(start, end) — no negative indexes
```

```
str.substring(6, 10); // "Test"
```

```
// at() for a single character
```

```
str.at(0); // "L"
```

```
str.at(-1); // "1"
```

Transforming Strings

These methods return a **new** string. The original never changes (strings are immutable).

```
let str = " Hello, World! ";
```

```
console.log(str.toUpperCase()); // " HELLO, WORLD! "
```

```
console.log(str.toLowerCase()); // " hello, world! "
```

```
// Trim removes spaces from the edges
```

```
console.log(str.trim()); // "Hello, World!"
```

```
str.trimStart();
```

```
str.trimEnd();
```

```
// Replace text
```

```
let msg = "Test: FAIL. Retry: FAIL.";
```

```
msg.replace("FAIL", "PASS"); // only the FIRST match
```

```
msg.replaceAll("FAIL", "PASS"); // every match
```

```
msg.replace(/FAIL/g, "PASS"); // every match using a regex (g = global)
```

```
// Join text together
```

```
"Hello" + " " + "World"; // "Hello World"
```

```
"Hello".concat(" ", "World"); // "Hello World"
```

```
`${"Hello"} ${"World"}`; // "Hello World"
```

```
// Splitting and joining
```



```
"pass,fail,skip".split(","); // ["pass", "fail", "skip"]
```

```
"hello".split(""); // ["h","e","l","l","o"]
```

```
"test_login_pass".split("_").join(" "); // "test login pass"
```

```
let parts = ["2024", "03", "07"];
```

```
let date = parts.join("-");
```

```
console.log(date); // "2024-03-07"
```

`split` turns a string into an array, and `join` turns an array back into a string. They are a great pair.



String Conversion and Immutability

```
// Convert other types to a string
```

```
(200).toString(); // "200"
```

```
true.toString(); // "true"
```

```
// Convert a string to a number
```

```
Number("42"); // 42
```

```
parseInt("42px"); // 42 (reads the number, ignores "px")
```

```
parseFloat("3.14rem"); // 3.14
```

Strings are **immutable**: you cannot change a single character in place. Trying to do so is silently ignored.

```
let str = "hello";
```

```
str[0] = "H"; // ignored, strings cannot be edited in place
```

```
console.log(str); // "hello"
```

```
let upper = str.toUpperCase();
```

```
console.log(str); // "hello" (original unchanged)
```

```
console.log(upper); // "HELLO" (a new string)
```

In Simple Terms



- Strings are text in quotes, backticks allow ``{ }`` and multiple lines
 - `includes`, `startsWith`, `endsWith` check content, `indexOf` finds position
 - `slice` and `substring` extract parts (`slice` allows negatives)
 - `toUpperCase`, `trim`, `replace`, `split`, `join` transform text
 - Strings are immutable, every "change" returns a brand new string
-

Chapter 11: Objects

What is an Object?

An object stores data as **key and value** pairs. Where an array is an ordered list, an object is a labelled collection: each value has a name (the key) instead of a number index. Objects are how we model real things like a user, a config, or an API response.

// Key and value

```
let student1 = { name: "Amit", age: 65 };
```

```
let student2 = { name: "Pramod" };
```

```
let student3 = { name: "Pramod", age: 87, phone: 987654320 };
```

// In plain JS the key is written without quotes.

// When the key is in quotes, that is JSON style.

```
let JSON_student4 = { "name": "Pramod", "age": 87, "phone": 987654320 };
```

You can read a value two ways: dot notation `obj.key`, or bracket notation `obj["key"]`.

```
let a = { status: "pass" };
```

```
console.log(a.status); // "pass"
```

```
console.log(a["status"]); // "pass"
```

// Keys are case sensitive: status and Status are different

```
let a22 = { status: "pass", Status: "fail" };
```

```
console.log(a22["status"]); // "pass"
```

```
console.log(a22["Status"]); // "fail"
```



Accessing and Modifying Properties

Use bracket notation when the key is stored in a variable (dynamic access). You can add, change, and delete properties at any time.

```
const user = {  
  
  name: "John",  
  
  age: 30,  
  
  email: "john@example.com"  
  
};  
  
console.log(user.name); // "John"  
  
console.log(user["age"]); // 30  
  
// Dynamic property access  
  
const key = "age";  
  
console.log(user[key]); // 30  
  
// Add or modify  
  
user.city = "NYC"; // add a new key  
  
user.age = 31; // change an existing key  
  
console.log(user);  
  
let config = {};  
  
config.browser = "Chrome";  
  
config.timeout = 3000;  
  
config.timeout = 5000; // overwrite with the latest value  
  
console.log(config);  
  
delete config.browser; // remove a key  
  
console.log(config); // { timeout: 5000 }
```



Primitives vs References (Very Important)

Primitive types (number, string, boolean, null, undefined) are copied **by value**. Objects and arrays are copied **by reference**, meaning two variables can point to the same object in memory.

// Primitive — copied by value (independent copies)

```
let a = 10;
```

```
let b = a;
```

```
b = 99;
```

```
console.log(a); // 10 (a did not change)
```

```
console.log(b); // 99
```

// Object — copied by reference (same object!)

```
let obj1 = { val: 10 };
```

```
let obj2 = obj1; // obj2 points to the SAME object
```

```
obj2.val = 99;
```

```
console.log(obj1.val); // 99 (obj1 changed too)
```

```
let a = { status: "pass" };
```

```
let b = a; // b copies the REFERENCE, not the object
```

```
b.status = "fail";
```

```
console.log(a.status); // "fail"
```

// Two separate objects are never equal, even with the same content

```
let c = { status: "pass" };
```

```
let d = { status: "pass" };
```

```
console.log(c === d); // false
```

This reference behaviour is the source of many bugs. Two objects are `===` only when they are literally the same object.



Object Methods and `this`

A function stored inside an object is called a **method**. Inside a method, the keyword `this` refers to the object it belongs to.

```
const calculator = {  
  
  value: 0,  
  
  add(n) {  
  
    this.value += n;  
  
    return this; // returning this allows chaining  
  
  },  
  
  subtract(n) {  
  
    this.value -= n;  
  
    return this;  
  
  }  
};  
  
console.log(calculator.add(5).subtract(6)); // value becomes -1  
  
const user = {  
  
  name: "Pramod",  
  
  saymyName(lastName) {  
  
    this.name += lastName;  
  
    return this.name;  
  
  }  
};  
  
console.log(user.saymyName("Dutta")); // "PramodDutta"
```

Because each method returns `this`, you can chain calls like `add(5).subtract(6)`.



Object Destructuring

Destructuring pulls values out of an object into variables in one line. You can rename them, set defaults, and even reach into nested objects.

```
const user = { name1: "John", age: 30, city: "NYC" };
```

```
// Basic
```

```
const { name1, age } = user;
```

```
console.log(name1); // "John"
```

```
console.log(age); // 30
```

```
// Rename while destructuring
```

```
const { name1: userName, age: userAge } = user;
```

```
console.log(userName); // "John"
```

```
// Default value if the key is missing
```

```
const { country = "USA" } = user;
```

```
console.log(country); // "USA"
```

```
// Nested destructuring
```

```
const data = { user: { name: "John", address: { city: "NYC" } } };
```

```
const { user: { address: { city } } } = data;
```

```
console.log(city); // "NYC"
```

Spread and Merging Objects

The spread operator `...` copies the properties of one object into another. It is the modern way to copy or merge objects.

```
const obj1 = { a: 1, b: 2 };
```

```
const obj2 = { c: 3, d: 4 };
```

```
const copy = { ...obj1 };
```

```
console.log(copy); // { a: 1, b: 2 }
```



```
const merged = { ...obj1, ...obj2 };

console.log(merged); // { a: 1, b: 2, c: 3, d: 4 }
```

■ Getters and Setters

A getter looks like a property but runs a function when you read it. A setter runs when you assign to it. They let an object compute a value on the fly.

```
const user = {

  firstName: "Pramod",

  lastName: "Dutta",

  get fullName() {

    return this.firstName + this.lastName;

  },

  set fullName(value) {

    [this.firstName, this.lastName] = value.split(" ");

  }

};

console.log(user.fullName); // "PramodDutta" (getter runs)

user.fullName = "Amit Sharma"; // setter runs, splits into first/last

console.log(user.fullName); // "AmitSharma"
```

■ Looping Over Objects

To walk through an object, use `Object.keys`, `Object.values`, or `Object.entries`, or a `for...in` loop.

```
const obj = { a: 1, b: 2, c: 3 };

console.log(Object.keys(obj)); // ["a", "b", "c"]

console.log(Object.values(obj)); // [1, 2, 3]
```



```
console.log(Object.entries(obj)); // [["a",1],["b",2],["c",3]]
```

```
const user = { name: "John", age: 30 };
```

```
// for...in gives each key
```

```
for (const key in user) {
```

```
    console.log(`${key}: ${user[key]}`);
```

```
}
```

```
// Object.entries with destructuring
```

```
Object.entries(user).forEach(([key, value]) => {
```

```
    console.log(`${key}: ${value}`);
```

```
});
```

Real Example: Config Objects

Objects are how testers store configuration and expected results. Nesting objects inside objects models real data.

```
const ENV = {
```

```
    BASE_URL: "https://staging.myapp.com",
```

```
    TIMEOUT: 5000,
```

```
    RETRIES: 2,
```

```
    BROWSER: "Chrome"
```

```
};
```

```
const EXPECTED_RESPONSE = {
```

```
    status: 200,
```

```
    body: {
```

```
        user: { role: "admin", active: true }
```

```
    }
```

```
};
```



```
const config = {  
  
  baseUrl: 'http://localhost:3000',  
  
  apiUrl: 'http://localhost:3000/api',  
  
  testUser: {  
  
    username: 'testuser@example.com',  
  
    password: 'SecurePass123',  
  
  },  
  
  logLevel: 'INFO',  
  
  retryCount: parseInt(process.env.RETRY_COUNT || '3', 10),  
  
};
```

let vs const with Objects

This is a common point of confusion. With **const**, you cannot **reassign** the variable to a new object, but you CAN still change the properties of the existing object.

```
const config = { browser: "Chrome", timeout: 3000 };  
  
// Modifying properties — ALLOWED  
  
config.browser = "Firefox";  
  
config.timeout = 5000;  
  
config.retries = 2;  
  
console.log(config); // works fine  
  
// config = { browser: "Safari" }; // ERROR — cannot reassign a const
```

With **let** you can do both: change properties and reassign to a whole new object.

In Simple Terms

- An object is a set of key and value pairs
- Read with **obj.key** or **obj["key"]**, keys are case sensitive
- Objects are copied by reference, **let b = a** shares the same object



- A method is a function on an object, `this` points to that object
 - Destructuring and spread (`...`) make copying and reading objects easy
 - `const` blocks reassignment, but object properties can still change
-

Chapter 12: Multi-Dimensional Arrays

What is a Multi-Dimensional Array?

A normal (1D) array is a single list. A multi-dimensional array is an **array of arrays**, like a table with rows and columns (a grid). The most common is the 2D array.

// 1D array — a simple list

```
let results = ["pass", "fail", "pass"];
```

// 2D array — array of arrays (like a table/grid)

```
let matrix = [
```

```
  [1, 2, 3],
```

```
  [4, 5, 6],
```

```
  [7, 8, 9]
```

```
];
```

Each inner array is a **row**, and each item inside a row is a **column**.

Accessing and Modifying (row, col)

You reach an item with two indexes: `grid[row][col]`. Both start at 0.

```
let grid = [
```

```
  [10, 20, 30],
```

```
  [40, 50, 60],
```

```
  [70, 80, 90]
```

```
];
```

```
console.log(grid[0][0]); // 10 (row 0, col 0)
```



```
// grid[2][1] -> 80

// grid[1][2] -> 60

grid[0][0] = 99; // modify one cell

console.log(grid[0][0]); // 99

console.log(grid.length); // 3 — number of rows

console.log(grid[0].length); // 3 — number of columns in row 0

// Last element = last row, last column

console.log(grid[grid.length - 1][grid[0].length - 1]); // 90

-   grid.length is the number of rows
-   grid[0].length is the number of columns in the first row
```

Looping Over a 2D Array

You need a loop inside a loop: the outer loop walks the rows, the inner loop walks the columns.

```
let testMatrix = [

  ["login", "pass", 200],

  ["checkout", "fail", 404],

  ["search", "pass", 180]

];

// Nested for loop (full index control)

for (let i = 0; i < testMatrix.length; i++) {

  for (let j = 0; j < testMatrix[i].length; j++) {

    console.log(testMatrix[i][j] + " ");

  }

}
```



```
// for...of (cleaner — gives the row, then the cell)
```

```
for (let row of testMatrix) {  
  for (let cell of row) {  
    process.stdout.write(cell + " ");  
  }  
  console.log();  
}
```

```
// forEach
```

```
testMatrix.forEach(row => {  
  row.forEach(cell => process.stdout.write(cell + " "));  
  console.log();  
});
```

■ Common 2D Operations

You can combine **map** and **reduce** to work on rows. For example, summing every student's scores:

```
let scores = [  
  [85, 90, 78], // student 0 -> 253  
  [60, 45, 70], // student 1 -> 175  
  [95, 88, 92]  // student 2 -> 275  
];  
  
let rowSums = scores.map(row => row.reduce((a, b) => a + b, 0));  
  
console.log(rowSums); // [253, 175, 275]
```

A testing example, finding every failed test across all suites:

```
let suiteResults = [
```



```
["login-pass", "register-pass", "logout-pass"], // Auth suite

["search-pass", "filter-fail", "sort-pass"],    // Search suite

["checkout-fail", "payment-fail", "confirm-pass"] // Payment suite

];

for (let i = 0; i < suiteResults.length; i++) {

  for (let j = 0; j < suiteResults[i].length; j++) {

    if (suiteResults[i][j].includes("fail")) {

      console.log(suiteResults[i][j]);

    }

  }

}
```



Star Patterns (Interview Practice)

Nested loops are the tool for printing patterns, a classic interview warm-up.

Right-angled triangle

```
// *

// * *

// * * *

let n = 3;

for (let i = 1; i <= n; i++) {

  let row = " ";

  for (let j = 1; j <= i; j++) {

    row += "* ";

  }

  console.log(row.trim());
```



```
}
```

Left-side descending triangle

```
// *****
```

```
// ****
```

```
// ***
```

```
// **
```

```
// *
```

```
let n = 5;
```

```
for (let i = n; i >= 1; i--) {
```

```
    let row = "";
```

```
    for (let j = 1; j <= i; j++) {
```

```
        row += "*";
```

```
    }
```

```
    console.log(row);
```

```
}
```

Pyramid

```
// *
```

```
// ***
```

```
// *****
```

```
let n = 3;
```

```
for (let i = 1; i <= n; i++) {
```

```
    let row = "";
```

```
    for (let j = 1; j <= n - i; j++) {
```

```
        row += " ";    // leading spaces
```



```
}  
  
for (let j = 1; j <= 2 * i - 1; j++) {  
  
    row += " ";    // stars  
  
}  
  
console.log(row);  
  
}
```

The idea in every pattern: the outer loop controls the rows, and the inner loop(s) control how many spaces and stars each row gets.

In Simple Terms

- A 2D array is a grid, an array of arrays
- Access a cell with `grid[row][col]`, both start at 0
- `grid.length` is rows, `grid[0].length` is columns
- Use a nested loop (loop inside a loop) to visit every cell
- Star patterns are just nested loops printing spaces and stars

Chapter 13: Callbacks

What is a Callback?

A callback is a function you pass into another function as an argument, so it can be called **later**, usually after some work is done. Callbacks are everywhere in JavaScript and in test frameworks like Playwright and Cypress.

// Callback

```
function placeOrder(item, callback) {  
  
    console.log("Placing order");  
  
    callback(); // the passed-in function is called here  
  
}  
  
function print() {  
  
    console.log("Normal Fn - Done with the order");  
  
}
```



```
}
```

```
placeOrder("Burger", print);
```

■ Three Ways to Pass a Callback

You can pass a named function, an anonymous function, or an arrow function. All three do the same thing.

```
// 1. Named function
```

```
placeOrder("Burger", print);
```

```
// 2. Anonymous function (no name)
```

```
placeOrder("Burger", function () {
```

```
    console.log("Anoy Fn, I am also a function wihtout name!");
```

```
});
```

```
// 3. Arrow function
```

```
placeOrder("Burger", () => {
```

```
    console.log("Arrow Fn, I am also a function wihtout name!");
```

```
});
```

This is exactly how test frameworks work. When you write `test('has title', async ({ page }) => { ... })`, the arrow function is a **callback** that the `test` function runs for you.

```
function test(text, callback) {
```

```
    console.log("Hi, this is test");
```

```
    callback();
```

```
}
```

```
test("Verify that the login page is working", async (page) => {
```

```
    console.log("Running TC1");
```

```
});
```



■ Callbacks with Parameters

A callback can receive data from the function that calls it.

```
function runTest(testName, callback) {  
  
    let status = "PASS";  
  
    callback(testName, status); // pass values into the callback  
  
}  
  
runTest("Login Test", function (name, result) {  
  
    console.log(name + " → " + result); // Login Test → PASS  
  
});
```

■ Synchronous Callbacks

A synchronous callback runs immediately and finishes before the next line. `forEach` is a synchronous callback.

```
let testResults = ["PASS", "FAIL", "PASS", "SKIP"];  
  
testResults.forEach(function (result, index) {  
  
    console.log("Test" + index + " -> " + result);  
  
});  
  
// forEach finishes all 4 iterations first, then the code moves on.  
  
let bugs = ["UI glitch", "API timeout", "Wrong redirect"];  
  
bugs.forEach(function (bug, i) {  
  
    console.log("Bug #" + (i + 1) + ": " + bug);  
  
});  
  
console.log("Total bugs: " + bugs.length);
```

■ Asynchronous Callbacks

An asynchronous callback runs **later**, after the current code has finished. `setTimeout` is the classic example. Notice the order of the output.



```
console.log("Test 1: started");

setTimeout(function () {

    console.log("Test 2 : API response received!");

}, 2000);

console.log("Test 3: Moving to next last");
```

The output is **Test 1**, then **Test 3**, and only after 2 seconds **Test 2**. JavaScript does not wait, it schedules the callback and keeps going. This is the heart of asynchronous programming.

Callback Hell (Pyramid of Doom)

When one async step depends on the previous one, callbacks start nesting inside callbacks. This deep, sideways-growing shape is called **callback hell** or the **pyramid of doom**.

// Real QA Scenario: E2E Login Flow

```
function openBrowser(callback) {

    console.log("opening the browser");

    setTimeout(function () {

        console.log("Step 1 - browser starting...");

        callback();

    }, 500);

}

function goToLoginPage(callback) {

    setTimeout(function () {

        console.log("Step 2: Login page loaded");

        callback();

    }, 500);

}
```



```
function enterCredentials(callback) {  
    setTimeout(function () {  
        console.log("Step 3: Credentials entered");  
        callback();  
    }, 500);  
}
```

```
function clickLogin(callback) {  
    setTimeout(function () {  
        console.log("Step 4: Login button clicked");  
        callback();  
    }, 500);  
}
```

```
// THIS IS CALLBACK HELL
```

```
openBrowser(function () {  
    goToLoginPage(function () {  
        enterCredentials(function () {  
            clickLogin(function () {  
                console.log("Test Complete!");  
            });  
        });  
    });  
});
```

This code works, but it is hard to read and hard to handle errors in. The solution is **Promises** (next chapter) and **async/await**, which flatten this pyramid into clean, top-to-bottom steps.



In Simple Terms

- A callback is a function passed to another function to run later
 - You can pass named, anonymous, or arrow functions as callbacks
 - Synchronous callbacks (like `forEach`) run right away
 - Asynchronous callbacks (like `setTimeout`) run later, JavaScript does not wait
 - Deeply nested callbacks are "callback hell", Promises and `async/await` fix it
-

Chapter 14: Promises

What is a Promise?

A Promise is an object that represents a value that will be available **later**. It is like ordering food: you get a promise that the food will come, and it will either succeed (resolve) or fail (reject). A promise is always in one of three states: **pending**, **fulfilled** (resolved), or **rejected**.

```
let order = new Promise(function (resolve, reject) {  
  
    let foodready = true;  
  
    if (foodready) {  
  
        resolve("Pizza is delivered!"); // success  
  
    } else {  
  
        reject("Order Cancelled!"); // failure  
  
    }  
  
});  
  
console.log(order);  
  
// A Promise is an OBJECT that wraps a value that arrives later.
```

You create a promise with `new Promise`, which gives you two functions: `resolve` (call on success) and `reject` (call on failure).



.then() — Handling Success

`.then()` runs only when the promise resolves successfully. The value you passed to `resolve` becomes the argument.

```
let apiCall = new Promise(function (resolve, reject) {  
  
    resolve({ status: 200, body: "User Data" });  
  
});  
  
apiCall.then(function (response) {  
  
    console.log(response);    // { status: 200, body: "User Data" }  
  
    console.log(response.status); // 200  
  
    console.log(response.body); // "User Data"  
  
});
```

.catch() — Handling Errors

`.catch()` runs only when the promise is rejected. When there is an error, `.then()` is skipped entirely and control jumps to `.catch()`.

```
let apiCall = new Promise(function (resolve, reject) {  
  
    reject("500 Error");  
  
});  
  
apiCall.then(function (data) {  
  
    console.log("Data is success!!"); // skipped  
  
}).catch(function (error) {  
  
    console.log(error); // "500 Error"  
  
});
```

.finally() — Always Runs

`.finally()` runs no matter what, whether the promise resolved or rejected. It is perfect for cleanup, just like `afterEach()` in Playwright or Cypress.



```
let testRun = new Promise(function (resolve, reject) {  
    reject("Assertion Failed");  
});  
  
testRun.then(function (data) {  
    console.log(data);  
}).catch(function (error) {  
    console.log(error); // "Assertion Failed"  
}).finally(function () {  
    console.log("I will be executed anyhow!!"); // always runs  
});
```

Promise Chaining (Real Login Flow)

Each `.then()` can `return` another promise, and the next `.then()` waits for it. This chains steps in a clean, top-to-bottom order and replaces callback hell.

```
function openBrowser() {  
    return new Promise(function (resolve) { resolve("Browser opened!"); });  
}  
  
function goToLogin() {  
    return new Promise(function (resolve) { resolve("Login page loaded"); });  
}  
  
function enterCredentials() {  
    return new Promise(function (resolve) { resolve("Credentials entered"); });  
}  
  
function clickLogin() {  
    return new Promise(function (resolve) { resolve("Logged in successfully"); });  
});
```



```
}  
  
openBrowser()  
  
  .then(function (msg) {  
    console.log("Step 1", msg);  
    return goToLogin();  
  }).then(function (msg) {  
    console.log("Step 2 :", msg);  
    return enterCredentials();  
  }).then(function (msg) {  
    console.log("Step 3 :", msg);  
    return clickLogin();  
  }).then(function (msg) {  
    console.log("Step 4 :", msg);  
  }).catch(function (error) {  
    console.log("Error:", error);  
  }).finally(function () {  
    console.log("Done execution!");  
  });
```

One `.catch()` at the end catches an error from any step in the chain.

Promise.all — Wait for All (Fail Fast)

`Promise.all` runs many promises at the same time and waits for **all** of them. If even one rejects, the whole thing rejects immediately.

```
let checkAuth = Promise.resolve("Auth OK");
```

```
let checkDB = Promise.resolve("DB OK");
```



```
let checkCache = Promise.resolve("Cache OK");

Promise.all([checkAuth, checkDB, checkCache]).then(function (results) {

    console.log("All checks:", results); // ["Auth OK", "DB OK", "Cache OK"]

});

// If one rejects, .catch() runs and the others' results are lost

Promise.all([

    Promise.resolve("OK"),

    Promise.reject("DB DOWN"),

    Promise.resolve("OK")

])

    .then(function (r) { console.log(r); })

    .catch(function (err) { console.log("Failed:", err); }); // Failed: DB DOWN
```

Promise.allSettled — Wait for All (Never Fails)

`Promise.allSettled` also runs everything in parallel, but it waits for **every** promise to finish and reports each result, whether it passed or failed. This is exactly what a test report needs.

```
Promise.allSettled([

    Promise.resolve("Test A Passed!"),

    Promise.reject("Test B failed"),

    Promise.resolve("Test C passed")

]).then(function (results) {

    results.forEach(function (r, i) {

        console.log("Test " + (i + 1) + ":", r.status, "-", r.value || r.reason);

    });

});
```



Each result has a **status** of **"fulfilled"** or **"rejected"**, with the value in **r.value** or the error in **r.reason**.

Promise.race — First One Wins

Promise.race settles as soon as the **first** promise finishes, resolved or rejected. Useful for timeouts or picking the fastest server.

```
let fastServer = new Promise(function (resolve) {  
    setTimeout(function () { resolve("Fast 100ms"); }, 100);  
});  
  
let slowServer = new Promise(function (resolve) {  
    setTimeout(function () { resolve("Fast 500ms"); }, 500);  
});  
  
Promise.race([fastServer, slowServer]).then(function (winner) {  
    console.log("Winner:", winner); // whichever finished first  
});
```

Method	Waits for	If one fails
<code>Promise.all</code>	all to succeed	rejects immediately (fail fast)
<code>Promise.allSettled</code>	all to finish	still returns every result
<code>Promise.race</code>	the first to finish	that first result wins

In Simple Terms

- A Promise is a value that arrives later, it is pending, then fulfilled or rejected
 - `.then()` handles success, `.catch()` handles errors, `.finally()` always runs
 - Chaining `.then()` replaces callback hell with clean steps
 - `Promise.all` fails fast, `allSettled` reports everything, `race` takes the first
-



Chapter 15: Async / Await

What is async / await?

`async` and `await` are a cleaner way to work with promises. They let you write asynchronous code that **reads** like normal top-to-bottom code, with no `.then()` chains. This is the style used in almost every modern Playwright test.

- `async` marks a function as asynchronous. An async function **always returns a promise**.
- `await` pauses inside an async function until a promise settles, then gives you its value.

// async function ALWAYS returns a Promise

```
async function getTestResults() {
```

```
    return "Pass";
```

```
}
```

```
getTestResults().then(function (result) {
```

```
    console.log(result); // "Pass"
```

```
});
```

// await unwraps the value from a promise

```
async function runTest() {
```

```
    let result = await Promise.resolve("Login test passed");
```

```
    console.log(result);
```

```
    let result2 = await Promise.resolve("Dashboard test passed");
```

```
    console.log(result2);
```

```
}
```

```
runTest();
```



■ From .then() to async / await

The same logic can be written both ways. `await` replaces the `.then()` chain and is much easier to read.

```
// The old way with .then()
```

```
getToken()
```

```
  .then(function (token) {  
    return getUser(token);  
  })  
  .then(function (user) {  
    console.log(user);  
  });
```

```
// The same thing with async/await
```

```
async function run() {
```

```
  let token = await getToken();  
  let user = await getUser(token);  
  console.log(user);  
}
```

Here is the login flow from the Promise chapter, rewritten with async/await. Notice how it now reads like a simple list of steps.

```
async function runLoginFlow() {
```

```
  let msg1 = await openBrowser();  
  console.log("Step 1:", msg1);  
  let msg2 = await goToLogin();  
  console.log("Step 2:", msg2);  
  let msg3 = await enterCredentials();
```



```
console.log("Step 3:", msg3);

let msg4 = await clickLogin();

console.log("Step 4:", msg4);

}

runLoginFlow();
```

■ Error Handling with try / catch

With promises you use `.catch()`. With async/await you use a regular `try / catch / finally` block, exactly like normal JavaScript.

```
async function testAPI() {

  try {

    let result = await Promise.reject("503 Service Unavailable");

    console.log('Result', result);

  } catch (error) {

    console.log('Error', error); // 503 Service Unavailable

  } finally {

    console.log("Clean up!!"); // always runs

  }

}

testAPI();
```

`try / catch / finally` maps directly onto `.then()` / `.catch()` / `.finally()`, same logic, cleaner syntax.

■ Sequential Execution

When Step 2 needs the result of Step 1, you must run them one after another. Each `await` waits for the previous line to finish.

```
function apiCall(name) {
```



```
return new Promise(function (resolve) {  
    setTimeout(function () { resolve(name + " 200 Ok!"); }, 1000);  
});  
}
```

```
async function sequentialTest() {  
    console.log("Starting of the Test");  
  
    let start = Date.now();  
  
    let r1 = await apiCall("Login");    // waits 1s  
  
    console.log(r1);  
  
    let r2 = await apiCall("Dashboard"); // waits another 1s  
  
    console.log(r2);  
  
    let r3 = await apiCall("Report");    // waits another 1s  
  
    console.log(r3);  
  
    console.log("Time: ~" + (Date.now() - start) + "ms"); // ~3000ms  
}
```

```
sequentialTest();
```

Three sequential 1-second calls take about **3 seconds** in total, because each waits for the one before it.

Parallel Execution with Promise.all

If the steps do NOT depend on each other, run them together with `Promise.all` and one `await`. This is much faster.

```
async function parallelTest() {  
    console.log("Starting of the Test");  
  
    let start = Date.now();  
  
    let [r1, r2, r3] = await Promise.all([
```



```
    apiCall("Auth Service"),

    apiCall("User Service"),

    apiCall("Payment Service")

  ]);

  console.log(r1);

  console.log(r2);

  console.log(r3);

  console.log("Time: ~" + (Date.now() - start) + "ms"); // ~1000ms
}

parallelTest();
```

The same three 1-second calls now finish in about **1 second**, because they run at the same time. Rule of thumb: use sequential `await` when steps depend on each other, use `Promise.all` when they are independent.

Real QA Example: Retry a Flaky Test

A flaky test sometimes fails and sometimes passes. A common pattern is to retry it a few times using `async/await` in a loop.

```
let attempt = 0;

function flakyAPI() {

  attempt++;

  if (attempt < 3) {

    return Promise.reject("Attempt " + attempt + ": failed");

  }

  return Promise.resolve("Attempt " + attempt + ": success!");

}

async function retryTesting(maxRetries) {
```



```
for (let i = 1; i <= maxRetries; i++) {  
  try {  
    let result = await flakyAPI();  
    console.log('Pass Promise!, I will exit also', result);  
    // break; // add break to stop retrying once it passes  
  } catch (error) {  
    console.log('Fail Promise!', error);  
  }  
}  
}  
}  
  
retryTesting(5);
```

The first two attempts reject and are caught, the third resolves. Adding a **break** after success would stop the loop once the test passes.

In Simple Terms

- **async** makes a function return a promise, **await** waits for a promise's value
 - **async/await** is just a cleaner way to write **.then()** chains
 - Use **try / catch / finally** for errors instead of **.catch()**
 - Sequential **await** runs steps one by one (slower, use when they depend on each other)
 - **Promise.all** runs steps together (faster, use when they are independent)
-



Chapter 16: OOP (Classes and Modules)

Export and Import (Modules)

Modules let you split code across files and share it. You **export** something from one file and **import** it into another. Anything not exported stays private to its file.

JavaScript

```
// MODULE: Export/Import Systems
// FILE: testutil.js
export const BASE_URL = "https://app.vwo.com";
export function formatUpperCaseString(s) { return
s.toUpperCase(); }

// FILE: logger.js
export default function log(msg) { console.log(`[LOG] ${msg}`); }
export function log2(msg) { console.log(`[LOGS] ${msg}`); }

// FILE: main.js
import log from './logger.js'; // Default
Import
import { BASE_URL as url } from './testutil.js'; // Named
Import with Alias
log(`Target: ${url}`);
```

import log from './logger.js'; // default import, no curly braces

log("starting the test cases");

What is a Class?

A class is a blueprint for creating objects. Remember it as **CAB**: a Class contains Attributes (data) and Behaviour (methods). You then create real objects from that blueprint.

```
class Person {
```

```
    // Attributes (data)
```



```
name;

email;

salary;

// Behaviour (methods)

sleep() {}

eat() {}

}

// CAB -> Class contains Attribute, Behaviour
```

Constructor and Objects

The **constructor** runs automatically when you create an object with **new**. It sets up the initial attributes using **this**.

```
class Car {

    constructor(assigned_name) {

        this.name = assigned_name;

    }

    drive() {

        console.log("Driving the car " + this.name);

    }

}

let hyundai_car = new Car("i10"); // new creates an object

hyundai_car.drive();           // Driving the car i10
```

A testing example, a **TestCase** blueprint used to make many test objects:

```
class TestCase {

    constructor(name, status, priority) {
```



```
    this.name = name;

    this.status = status;

    this.priority = priority;
}

display() {

    console.log(this.name + " → " + this.status + " → " + this.priority);

}
}
```

```
let loginTest_ref = new TestCase("Login Test", "PASS", "P0");
```

```
let signupTest_ref = new TestCase("Signup Test", "FAIL", "P1");
```

```
loginTest_ref.display();
```

```
// A method is just a function that lives inside a class.
```

Each object made from the same class holds its own copy of the attributes:

```
class Browser {

    constructor(name) {

        this.name = name;

        this.isOpen = true;

        console.log(name + " launched");

    }

    startBrowser() { console.log("starting the browser"); }

}
```

```
let chrome = new Browser("Chrome");
```

```
let firefox = new Browser("Firefox");
```

```
console.log(chrome.isOpen); // true
```



Public and Private Fields (#)

By default, attributes are **public** (anyone can read them). A field marked with **#** is **private**, it can only be used inside the class. This protects sensitive data like API keys.

```
class Credentials {  
  
    #apiKey; // private  
  
    user;    // public  
  
    constructor(user, key) {  
  
        this.user = user;    // public  
  
        this.#apiKey = key; // private  
  
    }  
  
    pramodgetAuthHeader() {  
  
        return "Bearer " + this.#apiKey; // accessed inside the class  
  
    }  
  
}  
  
let cred = new Credentials("admin", "scret_key_1234");  
  
console.log(cred.user);           // "admin" (public, works)  
  
console.log(cred.pramodgetAuthHeader()); // "Bearer scret_key_1234"  
  
// cred.#apiKey -> error, cannot touch a private field from outside
```

The only way to reach a private field from outside is through a public method that the class provides.

Static Members

A **static** attribute or method belongs to the **class itself**, not to any single object. Static members are shared across all objects. You call them with **ClassName.method()**, not **object.method()**.

```
class TestRunner {  
  
    static totalTests = 0;
```



```
static passCount = 0;

constructor(name, passed) {

    this.name = name;

    TestRunner.totalTests++;

    if (passed) TestRunner.passCount++;

}

static summary() {

    return TestRunner.passCount + "/" + TestRunner.totalTests + " passed";

}

}

new TestRunner("Login", true);

new TestRunner("Signup", false);

new TestRunner("Cart", true);

new TestRunner("Checkout", true);

console.log(TestRunner.summary()); // "3/4 passed"
```

Here `totalTests` and `passCount` are shared counters that every object updates. `summary()` is static because it reports on the whole class, not one object.

Encapsulation (Getters and Setters)

Encapsulation means hiding data and exposing controlled access to it. You keep a field private with `#`, then provide a **getter** to read it and a **setter** to change it, so you can add rules.

```
class Person {

    #child1;

    #child2;

    constructor(name, ch1, ch2) {
```



```
this.name = name;

this.#child1 = ch1;

this.#child2 = ch2;

}

getChild1() { return this.#child1; }      // getter

setChild1(changed_name) { this.#child1 = changed_name; } // setter

}

let p = new Person("Pramod", "Vrad", "Jenny");

console.log(p.getChild1()); // "Vrad"

p.setChild1("VIRAD");

console.log(p.getChild1()); // "VIRAD"
```

The real power of a setter is adding a **rule** before changing the data. Here a bank balance can only be changed by a cashier:

```
class ICICI {

    #balance;

    constructor(name, balance) {

        this.#balance = balance;

        this.name = name;

    }

    getBalance() { return this.#balance; }

    setBalance(balance, isCashier) {

        if (isCashier) {

            this.#balance = balance;

        } else {
```



```
        console.log("Not allowed");
    }
}

let pramod = new ICICI("Pramod", 1000);

pramod.setBalance(10000000, false); // "Not allowed"

console.log(pramod.getBalance()); // 1000 (unchanged)

let pramod_father = new ICICI("Pramod", 2000);

pramod_father.setBalance(300000, true); // allowed

console.log(pramod_father.getBalance()); // 300000
```

In Simple Terms

- **export / import** share code between files, unexported things stay private
- A class is a blueprint (CAB: Class, Attributes, Behaviour)
- **constructor + new** create objects, **this** refers to the current object
- **#field** is private, plain fields are public
- **static** members belong to the class and are shared by all objects
- Encapsulation = private data + getters/setters that control access

Chapter 17: OOP Inheritance

What is Inheritance?

Inheritance lets one class reuse the attributes and methods of another. The class that gives is the **parent** (base/super) class, and the class that receives is the **child** (derived/sub) class. You connect them with the **extends** keyword. This is the foundation of the Page Object Model in test automation.

```
class BasePage {

    constructor(pageName) {

        this.pageName = pageName;
```



```
}

open() { console.log("Opening the page "); }

close() { console.log("Closing the page "); }

}

class LoginPage extends BasePage {

}

let page = new LoginPage();

page.open(); // Opening the page

page.close(); // Closing the page

// LoginPage never defined open() or close(), it inherited them from BasePage.
```

The super Keyword

When a child has its own **constructor**, it must call **super(...)** first to run the parent's constructor. **super** passes the needed values up to the parent.

```
class Animal {

  constructor(name) {

    this.name = name;

  }

  eat() { console.log(this.name + " is eating"); }

  sleep() { console.log(this.name + " is sleeping"); }

}

class Dog extends Animal {

  constructor(name, breed) {

    super(name);    // calls Animal's constructor

    this.breed = breed;

  }

}
```



```
}  
  
  bark() { console.log(this.name, " is barking!"); }  
  
}  
  
let dog = new Dog("Rex", "Labrador");  
  
dog.eat(); // Rex is eating (inherited)  
  
dog.sleep(); // Rex is sleeping (inherited)  
  
dog.bark(); // Rex is barking! (its own)  
  
console.log(dog.breed); // Labrador
```

Method Overriding

A child can replace a parent's method by defining a method with the same name. When you call it, JavaScript uses the version on the actual object. This is called overriding.

```
class BaseTest {  
  
  setup() { console.log("Base: open browser"); }  
  
}  
  
class APITest extends BaseTest {  
  
  setup() { console.log("APITest: open browser"); } // overrides parent  
  
}  
  
let test = new APITest();  
  
test.setup(); // "APITest: open browser"
```

Sometimes you want to **add** to the parent's method, not fully replace it. Call **super.methodName()** inside the override to run the parent version too.

```
class BaseTest {  
  
  setup() { console.log("Base: open browser"); }  
  
  teardown() { console.log("Base: close browser"); }  
  
}
```



```
class UITest extends BaseTest {  
  
  setup() {  
  
    super.setup();          // run the parent's setup first  
  
    console.log("UI: maximize window");  
  
  }  
  
  teardown() {  
  
    console.log("UI: take screenshot");  
  
    super.teardown();       // then run the parent's teardown  
  
  }  
}  
  
let test = new UITest();  
  
test.setup();  
  
// Base: open browser  
  
// UI: maximize window  
  
test.teardown();  
  
// UI: take screenshot  
  
// Base: close browser
```

Note the difference: `super()` calls the parent **constructor**, while `super.methodName()` calls a parent **method**.

Polymorphism

Polymorphism means "many forms": the same method name behaves differently depending on the object. You can loop over a mixed list of objects and call the same method, and each does its own version.

```
class TestCase {  
  
  execute() { console.log("Running generic test"); }  
}
```



```
}

class UnitTest extends TestCase {

    execute() { console.log("Running unit test — checking one function"); }

}

class APITest extends TestCase {

    execute() { console.log("Running API test — sending HTTP request"); }

}

class E2ETest extends TestCase {

    execute() { console.log("Running E2E test — opening browser"); }

}

let tests = [new UnitTest(), new APITest(), new E2ETest()];

tests.forEach(function (test) {

    test.execute(); // each object runs its OWN execute()

});
```

The same idea powers a Page Object Model, where each page has its own `verify()`:

```
class BasePage {

    verify() { console.log("Verifying base page"); }

}

class LoginPage extends BasePage {

    verify() {

        console.log("Verify: username field exists");

        console.log("Verify: password field exists");

        console.log("Verify: login button is visible");

    }

}
```



```
    }  
  }  
  
  class DashboardPage extends BasePage {  
  
    verify() {  
  
      console.log("Verify: welcome message shown");  
  
      console.log("Verify: sidebar menu loaded");  
  
    }  
  }  
  
  let pages = [new LoginPage(), new DashboardPage()];  
  
  pages.forEach(function (page) {  
  
    page.verify();  
  
    console.log("---");  
  
  });
```

Multi-Level Inheritance

Inheritance can go down several levels, like Grandfather to Father to Son. Each level adds something new, and the bottom class gets everything above it.

// BasePage -> AuthPage -> AdminPage

```
class BasePage {  
  
  constructor(name) { this.name = name; }  
  
  open() { console.log("[OPEN] " + this.name); }  
  
}  
  
class AuthPage extends BasePage {  
  
  login(user) { console.log("[LOGIN] " + user); }  
  
}  
  
class AdminPage extends AuthPage {
```



```
constructor() { super("Admin Panel"); }

manageUsers() { console.log("[ADMIN] Managing users"); }

}

let admin = new AdminPage();

admin.open();           // from BasePage

admin.login("superadmin"); // from AuthPage

admin.manageUsers();    // its own
```

Hierarchical Inheritance

Hierarchical inheritance is when several child classes extend the **same** parent, like two sons of one father.

```
class Father { }

class Son1 extends Father { }

class Son2 extends Father { }
```

Both **Son1** and **Son2** inherit everything from **Father**, but they are separate classes.

Multiple Inheritance (via Mixins)

JavaScript does not allow a class to extend two parents at once (**class C extends A, B** is a `SyntaxError`). The workaround is a **mixin**: a function that takes a base class and returns a new class with extra abilities added. You can stack several mixins.

```
// Mixin 1: adds logging

let LoggerMixin = function (Base) {

  return class extends Base {

    log(msg) { console.log("[Log] " + msg); }

  };

};

// Mixin 2: adds screenshot
```



```
let ScreenshotMixin = function (Base) {  
  
  return class extends Base {  
  
    takeScreenshot() { console.log("[SCREENSHOT] captured"); }  
  
  };  
  
};  
  
class TestCase {  
  
  constructor(name) { this.name = name; }  
  
  run() { console.log("Running: " + this.name); }  
  
}  
  
// Apply BOTH mixins on top of TestCase  
  
class SmartTest extends ScreenshotMixin(LoggerMixin(TestCase)) {  
  
  constructor(name) { super(name); }  
  
}  
  
let t = new SmartTest("Login Flow");  
  
t.run();           // from TestCase  
  
t.log("Test started"); // from LoggerMixin  
  
t.takeScreenshot(); // from ScreenshotMixin
```

Exporting Classes (Page Object Model)

In a real framework you keep each class in its own file, export it, and extend it across files. This is exactly how a Page Object Model is built.

```
// Basepage.js  
  
export class BasePage {  
  
  constructor(name) { this.name = name; }  
  
  open() { console.log("Opening " + this.name); }
```



```
}  
  
// LoginPage.js  
  
import { BasePage } from "../Basepage.js";  
  
export class LoginPage extends BasePage {  
  
    constructor() { super("Login Page"); }  
  
    login(user) { console.log(user + " logged in"); }  
  
}
```

```
// Test file  
  
import { LoginPage } from "../LoginPage.js";  
  
let page = new LoginPage();  
  
page.open(); // Opening Login Page (inherited)  
  
page.login("admin"); // admin logged in
```

In Simple Terms

- Inheritance shares a parent class's code with a child using **extends**
 - **super()** calls the parent constructor, **super.method()** calls a parent method
 - Overriding replaces a parent method, polymorphism = same method, different behaviour per object
 - Chains (multi-level) and shared parents (hierarchical) are both allowed
 - Multiple inheritance is faked with mixins
 - Exporting and extending classes across files is the Page Object Model
-



Chapter 18: TypeScript Basics

What is TypeScript?

TypeScript is JavaScript with **types** added. You tell the code what kind of value each variable, parameter, and return should be, and TypeScript catches mistakes **before** you run the code. Every Playwright automation project today is written in TypeScript, so this is a must-know.

```
let testName1: string = "Login Test";

// Plain JavaScript — no type safety

// function add(a, b) { return a + b; }

// TypeScript — a and b must be numbers, and it returns a number

function add_ts(a: number, b: number): number {

    return a + b;

}
```

The pattern is **name: type**. If you try `add_ts("hi", 5)`, TypeScript refuses to compile it.

Basic Types

You annotate a variable by writing a colon and the type after its name.

```
// Primitive types

let name: string = "John";

let age: number = 30;

let pi: number = 3.14;           // TS has no separate float, all numbers are "number"

let distance_to_moon: number = 398765434567;

let isActive: boolean = true;
```



```
let nothing: null = null;
```

```
let notDefined: undefined = undefined;
```

```
// Arrays (two ways to write them)
```

```
let numbers: number[] = [1, 2, 3];
```

```
let names: Array<string> = ["John", "Jane"];
```

Note there is no separate `float` type, whole numbers and decimals are both `number`.



any vs unknown

`any` turns off type checking for that value, so you lose all safety. `unknown` is the safer cousin: you must first check the type before you can use it.

```
// any — avoid it, it disables checking
```

```
let anything: any = "hello";
```

```
// unknown — safer, you must narrow the type first
```

```
let value: unknown = "hello";
```

```
if (typeof value === "string") {
```

```
    console.log("Hi"); // only allowed after the type check
```

```
}
```

Rule of thumb: prefer `unknown` over `any` whenever you are unsure of a type.



Function and Object Annotations

You can type function parameters, return values, arrow functions, and object shapes.

```
// Function annotations
```

```
function greet(name: string): string {
```

```
    return `Hello, ${name}!`;
```

```
}
```

```
// Arrow function annotations
```



```
const multiply = (a: number, b: number): number => a * b;
```

```
// Object annotations
```

```
let user: { name: string; age: number } = {  
  name: "John",  
  age: 30  
};
```

■ Special Return Types: void and never

void means the function returns nothing. **never** means the function never finishes normally (it always throws or loops forever).

```
// void — no return value
```

```
function sayHello(msg: string): void {  
  console.log(msg);  
}
```

```
// never — the function never returns
```

```
function throwError(message: string): never {  
  throw new Error(message);  
}
```

```
function infiniteLoop(): never {  
  while (true) { }  
}
```

■ Real QA Examples

Types make test helper functions self-documenting: you can see exactly what goes in and what comes out.

```
function buildEndpoint(base: string, path: string): string {
```



```
    return base + path;
}

function isSuccessCode(code: number): boolean {
    return code >= 200 && code < 300;
}

function logTestStep(step: string): void {
    console.log("[STEP] " + step);
}

console.log(buildEndpoint("https://api.com", "/users")); // https://api.com/users

console.log("200 is success:", isSuccessCode(200)); // true
console.log("404 is success:", isSuccessCode(404)); // false

Typed arrays and objects describe your test data precisely:

let statusCode: number[] = [200, 201, 404, 500];

let testSuites: string[] = ["Smoke", "Regression", "Sanity"];

let testResult: { name: string; status: string; duration: number } = {
    name: "Login Test",
    status: "PASS",
    duration: 1200
};

// Filter only the failed response codes, with full type safety

let responseCodes: number[] = [200, 201, 404, 500, 302, 403];

function getFailedCodes(codes: number[]): number[] {
    return codes.filter(function (code: number): boolean {
        return code >= 400;
    });
}
```



```
});  
  
}  
  
console.log("Failed codes:", getFailedCodes(responseCodes)); // [404, 500, 403]
```

In Simple Terms

- TypeScript = JavaScript + types, it catches bugs before you run the code
 - Annotate with **name: type** for variables, parameters, and returns
 - Prefer **unknown** over **any**, **any** removes all safety
 - **void** returns nothing, **never** never finishes normally
 - Types make test helpers and test data self-documenting
-

Chapter 19: TypeScript Interfaces

What is an Interface?

An interface is a **contract** that describes the shape of an object: which properties it must have and of what type. If an object breaks the contract, TypeScript reports the error immediately. In Playwright projects, interfaces describe API response shapes, so if the backend renames a field, TypeScript flags every test that used the old name.

```
interface TestCase {  
  
  id: number;  
  
  name: string;  
  
  status: string;  
  
  duration: number;  
  
}  
  
let test1: TestCase = {  
  
  id: 1,  
  
  name: "Login with valid credentials",
```



```
    status: "PASS",

    duration: 1500

};

console.log("TC-" + test1.id + ": " + test1.name + " → " + test1.status);
```

If you forget a property (say `duration`), TypeScript will not let the object compile.

Optional (?) and readonly Properties

A `?` marks a property as optional. `readonly` means the property can be set once but never changed afterward.

```
interface APIResponse {

    readonly statusCode: number; // cannot be changed after creation

    body: string;

    headers?: object;           // optional

    responseTime?: number;      // optional

}

let response: APIResponse = {

    statusCode: 200,

    body: '{"user": "admin"}',

};

console.log("Status:", response.statusCode);

console.log("Headers:", response.headers); // undefined, that is fine (optional)

interface Point {

    readonly x: number;

    readonly y: number;

}
```



```
const point: Point = { x: 10, y: 20 };
```

```
// point.x = 5; // Error, x is readonly
```

■ Method Signatures in an Interface

An interface can also describe methods (functions the object must provide). There are two ways to write them.

```
interface Calculator {  
  
    add(a: number, b: number): number;  
  
    subtract(a: number, b: number): number;  
  
    multiply: (a: number, b: number) => number; // alternative arrow syntax  
  
}  
  
const calc: Calculator = {  
  
    add: (a, b) => a + b,  
  
    subtract: (a, b) => a - b,  
  
    multiply: (a, b) => a * b  
  
};
```

■ Function-Type Interfaces (Test Hooks)

An interface can describe the shape of a **function** itself. This is handy for test hooks like `beforeEach` and `afterEach`.

```
interface TestHook {  
  
    (testName: string): void; // any function that takes a string and returns nothing  
  
}  
  
let beforeEachHook: TestHook = function (testName: string): void {  
  
    console.log("[BEFORE] Setting up: " + testName);  
  
};  
  
let afterEachHook: TestHook = function (testName: string): void {
```



```
    console.log("[AFTER] Tearing down: " + testName);  
  
};  
  
beforeEachHook("Login Test");  
  
afterEachHook("Login Test");
```

Extending Interfaces

One interface can **extends** another to inherit its properties and add more. This mirrors class inheritance and is perfect for a Page Object Model.

```
interface BasePage {  
  
    url: string;  
  
    title: string;  
  
}  
  
interface LoginPage extends BasePage {  
  
    usernameSelector: string;  
  
    passwordSelector: string;  
  
    loginButtonSelector: string;  
  
}  
  
interface FreeTrailPage extends BasePage {  
  
    usernameSelector: string;  
  
    submitButtonSelector: string;  
  
}  
  
let loginPage: LoginPage = {  
  
    url: "/login",  
  
    title: "Login Page",  
  
    usernameSelector: "#username",
```



```
passwordSelector: "#password",  
  
loginButtonSelector: "#login-btn"  
  
};
```

Both `LoginPage` and `FreeTrailPage` automatically require `url` and `title` from `BasePage`.

A Class that implements an Interface

A class can promise to follow an interface using `implements`. TypeScript then forces the class to provide every property and method the interface requires.

```
interface Executable {  
  
    name: string;  
  
    run(): void;  
  
    getStatus(): string;  
  
}  
  
class TestCase implements Executable {  
  
    name: string;  
  
    constructor(name: string) {  
  
        this.name = name;  
  
    }  
  
    run(): void {  
  
        console.log("[RUN] " + this.name);  
  
    }  
  
    getStatus(): string {  
  
        return "PASS";  
  
    }  
  
}
```



```
}  
  
let tc: Executable = new TestCase("Verify login redirect");  
  
tc.run();  
  
console.log("Call:", tc.getStatus());
```

Index Signatures

When you do not know the property names in advance, an index signature lets you allow any key of a certain type.

```
interface StringDictionary {  
  
    [key: string]: string; // any string key maps to a string value  
  
}  
  
const dict: StringDictionary = {  
  
    hello: "world",  
  
    foo: "bar"  
  
};
```

Real Example: Test Config and Bug Report

Interfaces shine when modelling real test data, giving you autocomplete and error checking.

```
interface TestConfig {  
  
    browser: string;  
  
    headless: boolean;  
  
    baseUrl: string;  
  
    timeout?: number;  
  
    retries?: number;  
  
}  
  
let ciConfig: TestConfig = {
```



```
    browser: "Chrome",

    headless: true,

    baseURL: "https://staging.app.com"

};

interface BugReport {

    id: number;

    title: string;

    severity: string;

    stepsToReproduce: string[];

}

function logBug(bug: BugReport): void {

    console.log("BUG- Report -> " + bug.id + " [" + bug.severity + "] " + bug.title);

    bug.stepsToReproduce.forEach(function (step: string, i: number) {

        console.log(" " + (i + 1) + ". " + step);

    });

}

logBug({

    id: 1,

    title: "VWO login is not working. ",

    severity: "High",

    stepsToReproduce: ["Step1: open app.vwo.com", "Step2: enter invalid creds", "Step3: verify the error message"]

});
```

In Simple Terms

- An interface is a contract describing an object's shape



- `?` makes a property optional, `readonly` locks it after creation
 - Interfaces can describe methods and even function shapes (test hooks)
 - `extends` inherits an interface, `implements` forces a class to follow one
 - Index signatures allow unknown keys of a fixed type
-

Chapter 20: TypeScript Enums

What is an Enum?

An enum (enumeration) is a set of named constants. Instead of scattering "magic" strings like `"PASS"` or `"chrome"` all over your tests, you define them once in an enum and reuse the names. This prevents typos and gives you autocomplete.

```
enum TestStatus {  
  
    Pass = "PASS",  
  
    Fail = "FAIL",  
  
    Skip = "SKIP",  
  
    Pending = "PENDING",  
  
    Blocked = "BLOCKED"  
  
}  
  
console.log(TestStatus.Pass); // "PASS"
```

Now you write `TestStatus.Pass` instead of remembering the exact string `"PASS"`.

Numeric Enums and Reverse Mapping

If you do not assign values, an enum auto-numbers its members starting at `0`. Numeric enums also support **reverse mapping**: you can get the name back from the number.

```
enum Severity {  
  
    Low,    // 0  
  
    Medium, // 1  
  
    High,   // 2  
  
}
```



```
Critical // 3

}

console.log(Severity.High); // 2

console.log(Severity[2]); // "High" (reverse mapping)

function needsImmediateAttention(severity: Severity): boolean {

    return severity >= Severity.High; // compare by their numeric order

}

console.log("Low urgent?", needsImmediateAttention(Severity.Low)); // false

console.log("Critical urgent?", needsImmediateAttention(Severity.Critical)); // true

Because the members are numbers in order, you can compare them with >=.
```

String Enums for Configuration

String enums are great for environment URLs and other fixed text values. They are clearer in logs because they hold a readable string, not a number.

```
enum Environment {

    Dev = "https://dev.api.com",

    Staging = "https://staging.api.com",

    QA = "https://qa.api.com",

    Prod = "https://api.com"

}

console.log(Environment.QA); // "https://qa.api.com"
```

Real Example: Browser Selection

A very common testing use of enums is choosing a browser, then acting on the choice with a `switch`.

```
enum Browser {
```



```
Chrome = "chrome",

Firefox = "firefox",

Safari = "safari",

Edge = "edge"

}

function launchBrowser(browser: Browser): void {

    switch (browser) {

        case Browser.Chrome:

            console.log("Launching Chromium (Chrome v120)");

            break;

        case Browser.Firefox:

            console.log("Launching Gecko (Firefox v115)");

            break;

        case Browser.Safari:

            console.log("Launching WebKit (Safari v17)");

            break;

        case Browser.Edge:

            console.log("Launching Chromium (Edge v120)");

            break;

    }

}

launchBrowser(Browser.Chrome);

launchBrowser(Browser.Safari);
```



Because `launchBrowser` only accepts a `Browser`, you can never pass an invalid value like `"chrome"` by mistake.

Real Example: HTTP Methods

Enums keep API request methods consistent across your whole test suite.

```
enum HTTPMethod {  
  
    GET = "GET",  
  
    POST = "POST",  
  
    PUT = "PUT",  
  
    DELETE = "DELETE"  
}  
  
function sendRequest(method: HTTPMethod, endpoint: string): void {  
  
    console.log(method + " " + endpoint + " → 200 OK");  
}  
  
sendRequest(HTTPMethod.GET, "/api/users");  
  
sendRequest(HTTPMethod.POST, "/api/users");  
  
sendRequest(HTTPMethod.DELETE, "/api/users/1");
```

In Simple Terms

- An enum is a named set of fixed constants, no more magic strings
 - Numeric enums auto-number from 0 and support reverse mapping (`Enum[2]`)
 - String enums hold readable values, great for URLs and config
 - Enums pair perfectly with `switch` for browser or environment selection
 - Using an enum as a parameter type blocks invalid values at compile time
-



Chapter 21: TypeScript Generics

What are Generics?

Generics let you write code that works with **any type** while still keeping full type safety. Instead of locking a function to `string` or `number`, you use a type placeholder, written as `<T>`, and the caller decides the real type. `T` is just a name (for "Type"), you could call it anything.

```
// Locked to string — only works for strings
```

```
function getString(name: string): string {  
    return "Amit";  
}
```

```
// Generic — works for ANY type, T is decided by the caller
```

```
function getFirstResults<T>(results: T[]): T {  
    return results[0]!;  
}
```

```
let firstCode = getFirstResults<number>([200, 404, 500]);
```

```
let firstTest = getFirstResults<string>(["Login", "Signup", "Cart"]);
```

```
console.log("First code:", firstCode); // 200 (typed as number)
```

```
console.log("First test:", firstTest); // "Login" (typed as string)
```

The `!` after `results[0]` is a non-null assertion, it tells TypeScript "trust me, this is not undefined". When you call with `<number>`, `T` becomes `number` and the return is a `number`. One function, reused for every type, with the correct type kept intact.

Generic Classes

A class can be generic too. This is perfect for a reusable data store that should hold any kind of test data while staying type-safe.

```
class Test数据存储<T> {  
    private items: T[] = [];
```



```
add(item: T): void {  
    this.items.push(item);  
}  
  
getFirst(): T {  
    return this.items[0];  
}  
  
getAll(): T[] {  
    return this.items;  
}  
  
count(): number {  
    return this.items.length;  
}  
}  
  
let codeStore = new TestDataStorage<number>(); // a store of numbers  
  
let testStore = new TestDataStorage<string>(); // a store of strings  
  
codeStore.add(200);  
codeStore.add(404);  
codeStore.add(500);  
testStore.add("Login Test");  
testStore.add("Checkout Test");  
  
console.log("Codes:", codeStore.getAll()); // [200, 404, 500]  
  
console.log("First code:", codeStore.getFirst()); // 200  
  
console.log("Test count:", testStore.count()); // 2
```



`codeStore` only accepts numbers and `testStore` only accepts strings, even though both come from the same class. If you try `codeStore.add("hi")`, TypeScript stops you.

■ Real Example: Generic API Response

APIs return different data shapes. A generic wrapper describes "a response with a status code and some data", where the data can be any type.

```
function wrapResponse<T>(statusCode: number, data: T): { statusCode: number; data: T } {  
    return { statusCode: statusCode, data: data };  
}
```

```
let userResp = wrapResponse<string>(200, "admin");
```

```
console.log(userResp); // { statusCode: 200, data: "admin" }
```

```
let flagResp = wrapResponse<boolean>(200, true);
```

```
console.log(flagResp); // { statusCode: 200, data: true }
```

```
let countResp = wrapResponse<number>(200, 42);
```

```
console.log(countResp); // { statusCode: 200, data: 42 }
```

The `data` field is correctly typed as string, boolean, or number depending on how you call it, all from a single function.

In Simple Terms

- Generics write one piece of code that works for any type, safely
 - `<T>` is a type placeholder the caller fills in (`<number>`, `<string>`, ...)
 - Generic functions, classes, and return types all keep the exact type
 - Great for reusable helpers like data stores and API response wrappers
 - Without generics you would either repeat code per type or lose type safety with `any`
-



Chapter 22: TypeScript Access Modifiers

Access Modifiers: public, private, protected

Access modifiers control who can use a class member. TypeScript gives you three:

- **public**: usable everywhere (the default)
- **private**: usable only inside the same class
- **protected**: usable inside the class and its child classes (but not from outside)

```
class APIClient {  
  
    public baseUrl: string; // anyone can read  
  
    private apiKey: string; // only inside APIClient  
  
    protected timeout: number; // inside APIClient and its children  
  
    constructor(baseUrl: string, apiKey: string, timeout: number) {  
  
        this.baseUrl = baseUrl;  
  
        this.apiKey = apiKey;  
  
        this.timeout = timeout;  
  
    }  
  
    private getAuthHeader(): string {  
  
        return "Bearer " + this.apiKey;  
  
    }  
  
    public sendRequest(path: string): void {  
  
        console.log("GET " + this.baseUrl + path);  
  
        console.log("Auth: " + this.getAuthHeader()); // private method used internally  
  
        console.log("Timeout: " + this.timeout + "ms");  
  
    }  
}
```



```
}
```

```
class UserAPIClient extends APIClient {
```

```
  getUsers(): void {
```

```
    // can read protected timeout and baseUrl (it is a child)
```

```
    console.log("Fetching users (timeout: " + this.timeout + "ms)");
```

```
    console.log("URL: " + this.baseUrl + "/users");
```

```
  }
```

```
}
```

```
let client = new APIClient("https://api.staging.com", "key_secret_123", 5000);
```

```
console.log("Base URL:", client.baseUrl); // public, works
```

```
client.sendRequest("/health");
```

```
// client.apiKey -> error, private
```

```
// client.timeout -> error, protected
```

protected is the key one for a Page Object Model: a base page keeps helpers protected so only the pages that extend it can use them.

```
class BasePage {
```

```
  protected baseUrl: string;
```

```
  constructor(url: string) { this.baseUrl = url; }
```

```
  protected navigate(path: string): void {
```

```
    console.log("Navigating to: " + this.baseUrl + path);
```

```
  }
```

```
}
```

```
class LoginPage extends BasePage {
```

```
  constructor() { super("https://app.staging.com"); }
```



```
login(user: string): void {  
  
    this.navigate("/login"); // allowed, LoginPage is a child  
  
    console.log("Typing " + user + " into #username");  
  
    console.log("Clicking #login-btn");  
  
}  
  
}  
  
let page = new LoginPage();  
  
page.login("admin");
```

readonly Properties

readonly lets a property be set once (usually in the constructor) and never changed again. Perfect for config values that must stay fixed during a test run.

```
class PlaywrightConfig {  
  
    readonly baseURL: string;  
  
    readonly timeout: number;  
  
    readonly retries: number;  
  
    constructor(url: string, timeout: number, retries: number) {  
  
        this.baseURL = url;  
  
        this.timeout = timeout;  
  
        this.retries = retries;  
  
    }  
  
    showConfig(): void {  
  
        console.log("URL: " + this.baseURL);  
  
        console.log("Timeout: " + this.timeout + "ms");  
  
    }  
  
}
```



```
}

let config = new PlaywrightConfig("https://staging.app.com", 30000, 2);

config.showConfig();

// config.baseURL = "https://other.com"; // Error, readonly
```

Abstract Classes

An **abstract** class is a base blueprint that cannot be created on its own. It can declare **abstract** methods, which have no body, forcing every child to provide its own version. This guarantees a consistent structure.

```
abstract class BaseTest {

    protected testName: string;

    constructor(testName: string) { this.testName = testName; }

    abstract setup(): void; // child MUST implement

    abstract execute(): void; // child MUST implement

    abstract teardown(): void; // child MUST implement

}

class UITest extends BaseTest {

    setup(): void { console.log(" Setup: launch browser"); }

    execute(): void { console.log(" Execute: click buttons, fill forms"); }

    teardown(): void { console.log(" Teardown: close browser"); }

}
```

// new BaseTest(...) -> error, you cannot create an abstract class directly

Type Assertion (as)

Sometimes you know more about a value's type than TypeScript does, for example an **unknown** value from an API. The **as** keyword tells TypeScript to treat it as a specific type.

```
let rawResponse: unknown = {
```



```
status: 200,

body: { user: "admin", role: "tester" }

};

interface UserResponse {

    status: number;

    body: { user: string; role: string };

}

let response = rawResponse as UserResponse; // "treat this as a UserResponse"

console.log("Status:", response.status);

console.log("User:", response.body.user);

console.log("Role:", response.body.role);
```

The override Keyword

When a child method replaces a parent method, marking it **override** makes your intent explicit and lets TypeScript catch mistakes (like a misspelled method name that would silently create a new method instead of overriding).

```
class BaseTest {

    setup(): void { console.log("[BASE] Open browser"); }

    teardown(): void { console.log("[BASE] Close browser"); }

}

class LoginTest extends BaseTest {

    override setup(): void {

        console.log("[LoginTest] Open browser");

        console.log("[LoginTest] Maximize");

    }

}
```



```
}  
  
class APITest extends BaseTest {  
  
    override setup(): void {  
  
        console.log("[APITest] No Browser!");  
  
    }  
  
}  
  
let test = new LoginTest();  
  
let apitest = new APITest();  
  
test.setup(); // [LoginTest] ...  
  
apitest.setup(); // [APITest] No Browser!
```

Decorators

A decorator is a special function, written with `@`, that adds behaviour to a class or method without changing its code inside. Playwright and many frameworks use them for things like logging and metadata.

Class decorator (adds a property to the class):

```
function Role(role: string) {  
  
    return function (constructor: Function) {  
  
        constructor.prototype.role = role;  
  
    };  
  
}
```

`@Role("admin")`

```
class Admin { }
```

```
const admin = new Admin();
```

```
console.log((admin as any).role); // "admin"
```

Method decorator (wraps a method to log every call):



```
function Log(target: any, methodName: string, descriptor: PropertyDescriptor) {

    const original = descriptor.value;

    descriptor.value = function (...args: any[]) {

        console.log(`Called ${methodName} with args:`, args);

        return original.apply(this, args);

    };

}

class Calculator {

    @Log

    add(a: number, b: number) {

        return a + b;

    }

}

const calc = new Calculator();

calc.add(2, 3); // logs: Called add with args: [2, 3]
```

In Simple Terms

- **public** (everywhere), **private** (same class), **protected** (class and children)
 - **readonly** locks a property after it is first set
 - An **abstract** class is a blueprint you cannot instantiate, its abstract methods must be implemented by children
 - **as** tells TypeScript to treat a value as a specific type
 - **override** makes a method replacement explicit and catches typos
 - Decorators (**@name**) add behaviour to classes and methods from the outside
-



Chapter 23: Playwright Fundamentals

■ What is Playwright?

Playwright is a modern end-to-end testing framework from Microsoft. It drives a real browser (Chromium, Firefox, or WebKit) to open pages, click, type, and check results, exactly like a real user. It is written in TypeScript, which is why the earlier chapters on types, classes, and async/await all lead here.

A Playwright project has a standard shape:

- `tests/` folder holds your `.spec.ts` test files
- `playwright.config.ts` holds the settings (browsers, retries, reporter)
- `npx playwright test` runs the tests
- `npx playwright show-report` opens the HTML report

■ The Config File

`playwright.config.ts` controls how your whole suite runs. You edit it once and every test follows those rules.

```
import { defineConfig, devices } from '@playwright/test';
```

```
export default defineConfig({
```

```
  testDir: './tests',
```

```
  // Run test files in parallel
```

```
  fullyParallel: true,
```

```
  // Fail the CI build if test.only was left in the code
```

```
  forbidOnly: !!process.env.CI,
```

```
  // Retry failed tests on CI only
```

```
  retries: process.env.CI ? 2 : 0,
```

```
  // Reporter that generates an HTML report
```

```
  reporter: 'html',
```



```
use: {  
  // Collect a trace when a test is retried, for debugging  
  trace: 'on-first-retry',  
},  
projects: [  
  {  
    name: 'chromium',  
    use: { ...devices['Desktop Chrome'] },  
  },  
  // Firefox and WebKit projects can be enabled here too  
],  
});
```

Key settings:

- `testDir`: where the tests live
- `fullyParallel`: run tests at the same time for speed
- `retries`: how many times to re-run a failed (flaky) test
- `reporter`: `'html'`: build a nice HTML report
- `projects`: which browsers to run against
- `trace`: capture a step-by-step trace for debugging

Your First Test

A test file imports `test` and `expect`, then describes each test with `test('name', async ({ page }) => { ... })`. The `page` is the browser tab Playwright gives you. Notice this is exactly the callback and `async/await` pattern from earlier chapters.

```
import { test, expect } from '@playwright/test';  
  
test('has title', async ({ page }) => {  
  await page.goto('https://playwright.dev/');  
  
  // Expect the page title "to contain" a substring
```



```
await expect(page).toHaveTitle(/Playwright/);

});
```

- `test(name, callback)`: defines one test, the arrow function is the callback
- `async / await`: every Playwright action returns a promise, so you `await` it
- `page.goto(url)`: open a URL
- `expect(...)`: make an assertion (the check that decides pass or fail)

Actions and Locators

A **locator** is how you find an element on the page. Playwright recommends user-facing locators like `getByRole`, which find elements the way a real user (or screen reader) would. Once you have a locator, you act on it (click, type) or assert on it.

```
import { test, expect } from '@playwright/test';

test('get started link', async ({ page }) => {

  await page.goto('https://playwright.dev/');

  // Find the "Get started" link by its role and click it

  await page.getByRole('link', { name: 'Get started' }).click();

  // Assert the Installation heading is now visible

  await expect(page.getByRole('heading', { name: 'Installation' })).toBeVisible();

});
```

Common actions: `.click()`, `.fill("text")`, `.check()`, `.selectOption(...)`.
Common assertions: `toHaveTitle(...)`, `toBeVisible()`, `toHaveText(...)`, `toHaveURL(...)`.

How It All Connects

Everything from the JavaScript and TypeScript chapters shows up in a Playwright test:

- **async / await**: every action is awaited
- **callbacks / arrow functions**: the test body is an arrow-function callback
- **objects**: options like `{ name: 'Get started' }` are objects
- **classes and inheritance**: the Page Object Model wraps pages into classes (a `LoginPage` extends `BasePage`)
- **types and interfaces**: describe your test data and page shapes



- **enums**: hold browsers, environments, and HTTP methods

Running Tests

`npx playwright test` # run all tests (headless)

`npx playwright test --headed` # watch the browser as it runs

`npx playwright test example.spec.ts` # run one file

`npx playwright show-report` # open the HTML report

`npx playwright codegen <url>` # record actions into test code

In Simple Terms

- Playwright drives a real browser to test web apps end to end
 - `playwright.config.ts` sets browsers, retries, and the reporter
 - A test is `test('name', async ({ page }) => { ... })`, an async callback with a `page`
 - `page.goto`, `getByRole().click()`, and `expect(...)` are your core tools
 - Prefer user-facing locators like `getByRole` over brittle CSS selectors
 - Every earlier chapter (async, callbacks, classes, types) comes together here
-